SLAM : Projet Agora sprint 6 demande 1

Authentification, autorisations et gestion des utilisateurs avec le composant Security

Auteur : *Yannick Narbey, d'après un tutoriel de Marianne Delio*
Source *: https://symfony.com/doc/current/security.html (composant Symfony Security)*

Pré-requis : Symfony : contrôleurs, routes, templates twig
Versions minimum à respecter :
- Apache 2.4.54.2, PHP 8.2.0, MySQL 8.0.31
- Symfony CLI 5.5.10, Symfony 6.3, composer 2.6.4 (*composer self-update pour actualiser composer)*

RAPPEL : La version PHP 7.4 provoque une erreur lors de l'utilisation de la commande make, respectez les versions préconisées.

TABLE DES MATIERES

1. Contexte et objectifs ____ *2*
2. Réalisation ____ *4*
2.1 Installation des modules Symfony complémentaires ____ 4
2.2 Création de l'entité et de la table Membre ____ 7
2.3 Créer des utilisateurs de test avec DoctrineFixturesBundle ____ 16
2.4 Gérer l'authentification ____ 21
2.4.1 Créer un formulaire de connexion ____ 21
2.4.1 Afficher le nom de l'utilisateur ____ 26
2.4.2 Gérer la déconnexion ____ 30
2.5 Gérer les membres ____ 32
2.6 Gérer les autorisations ____ 40
3. Conclusion ____ *42*

1. Contexte et objectifs

L'association Agora souhaite disposer d'une **interface pour gérer les comptes d'accès** à l'application et demande de pouvoir donner des **droits d'accès différents aux utilisateurs** en fonction de leur rôle dans l'association.

L'association continue de confier cette mission à l'ESN **Logma**. Le développement de l'application se fera dans un cadre de travail agile **SCRUM**.

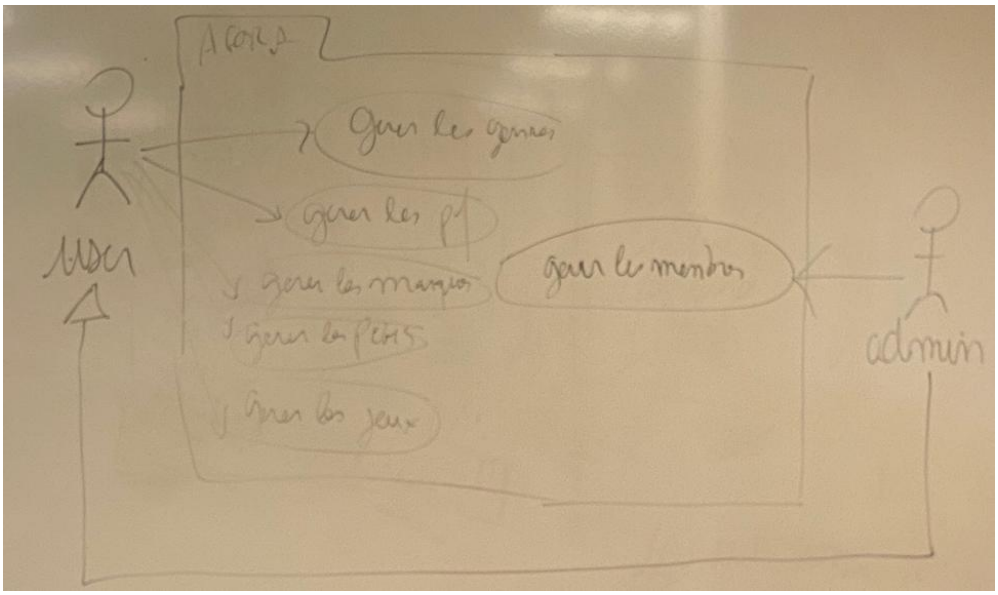
L'administration des données de l'application ne sera bien sûr pas accessible à tous les utilisateurs. Il convient de mettre en place un système d'authentification. M. Sarment, responsable administratif de l'association et Product Owner de ce projet, a demandé la **mise en place de l'authentification dans le sprint 6**.
Il a défini **2 profils d'utilisateurs** à gérer dans le back office :

- Administrateur : accès total
- Utilisateur : accès à tout sauf la gestion des utilisateurs

Pour développer le back office, c'est-à-dire l'application web d'administration des données, Logma va continuer à utiliser le framework **Symfony**. Il facilitera la prise en charge des évolutions futures déjà envisagées par l'association. La création de l'entité membre sur laquelle la gestion des comptes d'accès sera mené avec le composant doctrine.

Vous êtes chargé de faire évoluer l'application Symfony pour permettre de gérer les utilisateurs et la sécurité de la connexion avec les composants dédiés de Symfony.

Le concepteur de Logma a réalisé ce diagramme de cas d'utilisation qui permet de représenter les rôles utilisateur et administrateur et les fonctionnalités auxquels ils ont respectivement accès :



Ce tutoriel vous guide dans la mise en place du mécanisme d'authentification avec Symfony

Symfony (rappel)

Symfony est un ensemble de composants PHP, un **framework d'application Web**, une philosophie et une communauté - tous travaillant ensemble en harmonie. (site officiel)
Symfony est un framework basé sur le modèle MVC qui permet de réaliser des sites complexes rapidement, mais de façon structurée et avec un code clair et maintenable.
SensioLabs est le créateur du framework PHP Symfony. Le nom SensioLabs vient du mot espagnol "sencillo" qui signifie "simple". Fabien Potencier est le fondateur et chef de projet Symfony.

https://symfony.com/

Doctrine (rappel)

Symfony est livré par défaut avec l'**ORM** Doctrine. Un ORM (pour **Object-Relation Mapper**, « lien objet-relation ») est un programme qui se place en interface entre un programme applicatif et une base de données relationnelle pour simuler une base de données orientée objet. Il définit des **correspondances** entre les **schémas de la base** de données et les **classes** du programme applicatif.

https://www.doctrine-project.org

Entité (rappel)

Les **données** de la base de données sont des **objets**. Un objet dont l'enregistrement est confié à l'ORM se nomme une **entité** (entity en anglais). On dit également **persister une entité**, plutôt qu'enregistrer une entité.

Dans le projet Agora, nous avons déjà vu la notion d'authentification dans la mission 2.

Le mécanisme d'authentification

Le mécanisme d'authentification consiste à établir un lien irréfutable entre une personne et une identité numérique.

Un nouveau formulaire d'authentification permettra aux administrateurs de saisir leurs données d'identification des utilisateurs (email, mot de passe) pour leur permettre d'accéder à l'application. La gestion des données des utilisateurs (CRUD) sera également prise en charge par l'application.

Il est possible de coder soi-même toutes les fonctionnalités, mais comme il s'agit de **besoins récurrents, de plus liés à la sécurité**, il est très souvent plus judicieux d'utiliser des solutions existantes éprouvées (solutions qui ont fait leurs preuves, dont la valeur, la qualité sont reconnues). Dans le contexte de l'authentification, utiliser une solution éprouvée permet de bénéficier rapidement d'une solution sécurisée solide.

Nous allons donc utiliser 2 bundles (~package) de l'écosystème Symfony :

a) Le FOSUserBundle

Dans l'écosystème Symfony, le **FOSUserBundle** est de loin le bundle le plus utilisé. Il a été créé dès la sortie de Symfony2 par le groupe de développeurs FOS alias Friends Of Symfony.
Le FOSUserBundle permet de gérer les utilisateurs et leurs rôles. Il fournit un système de membres avec formulaire d'inscription, de connexion ainsi que de mot de passe oublié.
Il peut se configurer avec les **ORM** (*object-relational mapping*, soit mapping objet-relationnel) Propel, **Doctrine**, CouchDB et MongoDB. Il se combine également très bien avec le firewall de Symfony.

b) Le MakerBundle

Symfony définit son propre système de sécurité performant et pratique. Le développeur a le contrôle sur tout.
Il suffit d'utiliser le MakerBundle qui permet de générer du code pour certains cas courants. En ce qui concerne la gestion des utilisateurs, plusieurs commandes nous seront utiles :

- make:user pour générer une entité Membre (Doctrine) qui contiendra les champs requis par la sécurité,
- make:auth form-login pour créer un formulaire de connexion,
- make:registration-form pour réaliser un processus d'inscription (formulaire, contrôleur…).

Les trois fonctionnalités de base du FOSUserBundle sont donc couvertes et le code généré est plus simple.
Les différentes fonctions du MakerBundle sont listées avec : *php bin/console list make*

2. Réalisation

2.1 Installation des modules Symfony complémentaires

1. Les composants dont dépend l'application sont répertoriés dans le fichier **composer.json** placé à la racine de l'application. Pour les besoins du sprint 6, il faut ajouter les composants security :

Le **Security Bundle** est le composant officiel de Symfony dédié à la **gestion de la sécurité et de l'authentification** des applications web.

Il fournit un ensemble complet de fonctionnalités permettant de contrôler l'accès aux ressources, d'authentifier les utilisateurs et de gérer leurs rôles et permissions.
Les principales fonctionnalités du Security Bundle sont :

- **Authentification** : gestion des connexions utilisateurs via formulaire, HTTP Basic, JWT, ou autres fournisseurs (LDAP, OAuth, etc.) ;
- **Autorisation** : définition de **rôles** (ROLE_USER, ROLE_ADMIN…) et de **règles d'accès** aux routes et actions via security.yaml ou des annotations (@IsGranted, @Security) ;
- **Gestion des utilisateurs** : intégration avec une entité User (souvent reliée à Doctrine) et un système de hachage sécurisé des mots de passe (PasswordHasher ou UserPasswordHasherInterface) ;
- **Pare-feu (firewalls)** : configuration des zones sécurisées de l'application et des méthodes d'authentification associées ;
- **Accès et contrôle granulaire** : protection automatique contre les accès non autorisés, avec gestion des redirections vers la page de connexion ou d'erreur 403.

Le **Security Bundle** s'intègre naturellement au reste de l'écosystème Symfony et constitue la base pour mettre en place un système d'authentification robuste, extensible et conforme aux bonnes pratiques de sécurité web.

https://symfony.com/doc/current/security.html

Passez les commandes suivantes pour installer les composants complémentaires :

Security avant security -bundle :

```
C:\wamp64\www\agoram61>composer require security
```

```
C:\Windows\System32\cmd.e
C:\wamp64\www\agoram61>composer require security
./composer.json has been updated
Running composer update symfony/security-bundle
Loading composer repositories with package information
Restricting packages listed in "symfony/symfony" to "6.0.*"
Updating dependencies
Lock file operations: 2 installs, 0 updates, 0 removals
  - Locking symfony/security-bundle (v6.0.20)
  - Locking symfony/security-http (v6.0.20)
Writing lock file
Installing dependencies from lock file (including require-dev)
Package operations: 2 installs, 0 updates, 0 removals
  - Installing symfony/security-http (v6.0.20): Extracting archive
  - Installing symfony/security-bundle (v6.0.20): Extracting archive
Package doctrine/cache is abandoned, you should avoid using it. No replacement was suggested.
Package sensio/framework-extra-bundle is abandoned, you should avoid using it. Use Symfony instead.
Generating optimized autoload files
Class PdoJeux located in ./src/Controller/modele/class.PdoJeux.inc.php does not comply with psr-4 autoloading standard (
rule: App\ => ./src). Skipping.
67 packages you are using are looking for funding.
Use the `composer fund` command to find out more!

Symfony operations: 1 recipe (b728e38f527bc9282670684c7eeb9ed2)
  - Configuring symfony/security-bundle (>=6.0): From github.com/symfony/recipes:main
Executing script cache:clear [OK]
Executing script assets:install public [OK]

 What's next?


Some files have been created and/or updated to configure your new packages.
Please review, edit and commit them: these files are yours.

Found 10 security vulnerability advisories affecting 7 packages.
Run "composer audit" for a full list of advisories.
```

Ouvrir le fichier composer.json et vérifier que les modules sont installés

Fichier agoram61/composer.json

```
{
    "type": "project",
    "license": "proprietary",
    "minimum-stability": "stable",
    "prefer-stable": true,
    "require": {
        "php": ">=8.0.2",
        "ext-ctype": "*",
        "ext-iconv": "*",
        "doctrine/dbal": "^3",
        "doctrine/doctrine-bundle": "^2.13",
        "doctrine/doctrine-migrations-bundle": "^3.4",
        "doctrine/orm": "^2.20",
        "sensio/framework-extra-bundle": "^6.2",
        "symfony/asset": "6.0.*",
        "symfony/console": "6.0.*",
        "symfony/dotenv": "6.0.*",
        "symfony/flex": "^2",
        "symfony/form": "6.0.*",
        "symfony/framework-bundle": "6.0.*",
        "symfony/monolog-bundle": "^3.8",
        "symfony/runtime": "6.0.*",
        "symfony/security-bundle": "6.0.*",
        "symfony/security-csrf": "6.0.*",
        "symfony/twig-bundle": "6.0.*",
```

```
        "symfony/validator": "6.0.*",
        "symfony/yaml": "6.0.*"
    },
    "config": {
        "allow-plugins": {
...
```

2. Il faut vérifier la présence de vulnérabilités connues des composants dont dépend l'application (à relancer régulièrement) avec la commande : **symfony check:security**

```
C:\wamp64\www\agoram61>symfony check:security
Symfony Security Check Report
=============================

No packages have known vulnerabilities.

Note that this checker can only detect vulnerabilities that are referenced in the security advisories database.
Execute this command regularly to check the newly discovered vulnerabilities.
```

2.2 Création de l'entité et de la table Membre

1. Créer une classe Membre

Utiliser la commande **make:user** qui va créer la classe membre et permettre à symfony de s'appuyer dessus pour gérer l'authentification.

```
C:\wamp64\www\agoram61>php bin/console make:user
```

```
C:\Windows\System32\cmd.e
C:\wamp64\www\agoram61>php bin/console make:user

 The name of the security user class (e.g. User) [User]:
 > membre

 Do you want to store user data in the database (via Doctrine)? (yes/no) [yes]:
 >

 Enter a property name that will be the unique "display" name for the user (e.g. email, username, uuid) [email]:
 > username

 Will this app need to hash/check user passwords? Choose No if passwords are not needed or will be checked/hashed by som
e other system (e.g. a single sign-on server).

 Does this app need to hash/check user passwords? (yes/no) [yes]:
 >

 created: src/Entity/Membre.php
 created: src/Repository/MembreRepository.php
 updated: src/Entity/Membre.php
 updated: config/packages/security.yaml

 Success!

 Next Steps:
   - Review your new App\Entity\Membre class.
   - Use make:entity to add more fields to your Membre entity and then run make:migration.
   - Create a way to authenticate! See https://symfony.com/doc/current/security.html
```

Le nom de la classe représentant un utilisateur sur lequel se basera le système d'authentification est membre.
Le champ unique qui permettra d'identifier l'utilisateur est le "username".

La valeur par défaut permettant le chiffrement du mot de passe au moyen d'un algorithme de hashage a été validé.

2. Un fichier **src/Entity/Membre.php** a été créé. **Grâce aux annotations, chaque propriété est mappée** (mise en correspondance) avec une colonne de la table de la base de données. Exemple pour la propriété username : `#[ORM\Column(length: 180)]`. Les accesseurs (getters et setters) ont également été générés. Le code généré peut être modifié (ou développé) sans passer par la commande make qui n'est qu'une facilité.

Fichier agoram61/src/Entity/Membre.php

```php
<?php

namespace App\Entity;

use App\Repository\MembreRepository;
use Doctrine\ORM\Mapping as ORM;
use Symfony\Component\Security\Core\User\PasswordAuthenticatedUserInterface;
use Symfony\Component\Security\Core\User\UserInterface;

#[ORM\Entity(repositoryClass: MembreRepository::class)]
class Membre implements UserInterface, PasswordAuthenticatedUserInterface
{
    #[ORM\Id]
    #[ORM\GeneratedValue]
    #[ORM\Column]
```

```php
    private ?int $id = null;

    #[ORM\Column(length: 180)]
    private ?string $username = null;

    /**
     * @var list<string> The user roles
     */
    #[ORM\Column]
    private array $roles = [];

    /**
     * @var string The hashed password
     */
    #[ORM\Column]
    private ?string $password = null;

    public function getId(): ?int
    {
        return $this->id;
    }

    public function getUsername(): ?string
    {
        return $this->username;
    }

    public function setUsername(string $username): static
    {
        $this->username = $username;

        return $this;
    }

    /**
     * A visual identifier that represents this user.
     *
     * @see UserInterface
     */
    public function getUserIdentifier(): string
    {
        return (string) $this->username;
    }

    /**
     * @see UserInterface
     */
    public function getRoles(): array
    {
        $roles = $this->roles;
        // guarantee every user at least has ROLE_USER
        $roles[] = 'ROLE_USER';
```

```
        return array_unique($roles);
    }

    /**
     * @param list<string> $roles
     */
    public function setRoles(array $roles): static
    {
        $this->roles = $roles;

        return $this;
    }

    /**
     * @see PasswordAuthenticatedUserInterface
     */
    public function getPassword(): string
    {
        return $this->password;
    }

    public function setPassword(string $password): static
    {
        $this->password = $password;

        return $this;
    }

    /**
     * @see UserInterface
     */
    public function eraseCredentials(): void
    {
        // If you store any temporary, sensitive data on the user, clear it here
        // $this->plainPassword = null;
    }
}
```

Remarques-rappels :

```
class Membre implements UserInterface
```

La classe *Membre* hérite de *UserInterface*. .
UserInterface est une **interface** créée par Symfony qui définit juste quelques fonctions de bases

Rappel
Une **interface** correspond à une liste d'opérations assurant un service cohérent et constituant un contrat à respecter par les classes réalisant l'interface. Une interface s'apparente à une classe abstraite

```
public function getId(): ?int
```

PHP 7 ajoute le support des déclarations du type de retour. Les déclarations du type de retour spécifient le type de la valeur qui sera retournée par une fonction. Les mêmes types sont disponibles pour le typage du retour que pour le typage des arguments. Depuis PHP 7.1.0, les valeurs de retour peuvent être marquées comme nullable en préfixant le nom du type avec un point d'interrogation **(?). Ceci signifie que la fonction retourne soit le type spécifié soit NULL.**

```php
public function setEmail(string $email): static
```

Le nouveau type de retour statique (PHP 8) permet de réduire le type de retour à la classe appelée, ici Membre.

Le maker a également créé le *MembreRepository* . Un repository centralise tout ce qui touche à la récupération des entités à partir de la base de données. Pour respecter l'architecture, les requêtes SQL (ou leur équivalent, nous verrons cela par la suite) se trouvent donc uniquement dans les repository.

Fichier agoram61/src/Repository/MembreRepository.php

```php
<?php

namespace App\Repository;

use App\Entity\Membre;
use Doctrine\Bundle\DoctrineBundle\Repository\ServiceEntityRepository;
use Doctrine\Persistence\ManagerRegistry;
use Symfony\Component\Security\Core\Exception\UnsupportedUserException;
use Symfony\Component\Security\Core\User\PasswordAuthenticatedUserInterface;
use Symfony\Component\Security\Core\User\PasswordUpgraderInterface;

/**
 * @extends ServiceEntityRepository<Membre>
 */
class MembreRepository extends ServiceEntityRepository implements
PasswordUpgraderInterface
{
    public function __construct(ManagerRegistry $registry)
    {
        parent::__construct($registry, Membre::class);
    }

    /**
     * Used to upgrade (rehash) the user's password automatically over time.
     */
    public function upgradePassword(PasswordAuthenticatedUserInterface $user,
string $newHashedPassword): void
    {
        if (!$user instanceof Membre) {
            throw new UnsupportedUserException(sprintf('Instances of "%s" are not
supported.', $user::class));
        }

        $user->setPassword($newHashedPassword);
        $this->getEntityManager()->persist($user);
        $this->getEntityManager()->flush();
    }

//    /**
//     * @return Membre[] Returns an array of User objects
//     */
//    public function findByExampleField($value): array
//    {
```

```
//        return $this->createQueryBuilder('u')
//            ->andWhere('u.exampleField = :val')
//            ->setParameter('val', $value)
//            ->orderBy('u.id', 'ASC')
//            ->setMaxResults(10)
//            ->getQuery()
//            ->getResult()
//        ;
//    }

//    public function findOneBySomeField($value): ?Membre
//    {
//        return $this->createQueryBuilder('u')
//            ->andWhere('u.exampleField = :val')
//            ->setParameter('val', $value)
//            ->getQuery()
//            ->getOneOrNullResult()
//        ;
//    }
}
```

Le composant maker a également modifié le fichier /config/packages/security.yaml

Fichier agora5/config/packages/security.yml

```
security:
    # https://symfony.com/doc/current/secu
    password_hashers:
        Symfony\Component\Security\Core\User\PasswordAuthenticatedUserInterface: 'auto'
    # https://symfony.com/doc/current/security.html#loading-the-user-the-user-provider
    providers:
        # used to reload user from session & other features (e.g. switch_user)
        app_user_provider:
            entity:
                class: App\Entity\Membre
                property: username
    firewalls:
        dev:
            pattern: ^/(_(profiler|wdt)|css|images|js)/
            security: false
        main:
            lazy: true
            provider: app_user_provider

            # activate different ways to authenticate
            https://symfony.com/doc/current/security.html#the-firewall

            m/doc/current/security/impersonating_user.html


    # Easy way to control access for large sections of your site
    # Note: Only the *first* access control that matches will be used
    access_control:
```

configure l'encodage du mot de passe. La valeur "auto" sélectionne le meilleur algorithme de hachage disponible. voir : https://symfony.com/doc/current/security/passwords.html#the-auto-

le **User Provider** permet de récupérer les informations de l'utilisateur depuis une **session** User Provider utilise la classe *Membre* , l'attribut *username* servira d'identifiant pour la connexion

Le pare-feu dev est un faux pare-feu : il s'assure que vous ne bloquez pas accidentellement les outils de développement de Symfony - qui correspondent à des Urls comme /_profiler et /_wdt

toute requête entrante passe par le pare-feu main qui définit l'accès à l'application

On y définit les autorisations. Nous compléterons cette partie par la suite. Ainsi un utilisateur non authentifié pourra seulement accéder à la page

```
        # - { path: ^/admin, roles: ROLE_ADMIN }
        # - { path: ^/profile, roles: ROLE_USER }

when@test:
    security:
        password_hashers:
            # By default, password hashers are resource intensive and take time.
This is
            # important to generate secure password hashes. In tests however,
secure hashes
            # are not important, waste resources and increase test times. The
following
            # reduces the work factor to the lowest possible values.
            Symfony\Component\Security\Core\User\PasswordAuthenticatedUserInterfac
e:
                algorithm: auto
                cost: 4 # Lowest possible value for bcrypt
                time_cost: 3 # Lowest possible value for argon
                memory_cost: 10 # Lowest possible value for argon
```

La section "**firewalls**" est la plus importante. Elle définit quelles parties de l'application sont sécurisées et comment les utilisateurs peuvent s'authentifier. Elle va permettre de définir le système d'authentification de l'application.
https://symfony.com/doc/current/security.html#the-firewall

`main:`
`  lazy: true`
`  provider: app_user_provider`
	Le mode lazy anonymous permet d'accéder au site sans être authentifié (page de connexion dans ce cas pour notre application).
	La clé *provider* définie à app_user_provider indique que le mode d'authentification utilisera ce provider «entity membre» défini plus haut.

Apres l'installation du composant Security, la création de l'entity membre et son paramétrage comme APP_USER_PRODIDER dans le firewall, la debugbar s'est enrichie d'une icône supplémentaire.

3. Comme il faudra afficher le nom de l'utilisateur en clair, nous complétons notre classe membre avec nom, prénom et téléphone. On utilise à nouveau la commande make :

```
C:\wamp64\www\agoram61>php bin/console make:entity

 Class name of the entity to create or update (e.g. GrumpyPizza):
 > membre
```

```
Your entity already exists! So let's add some new fields!

New property name (press <return> to stop adding fields):
> nomMembre

Field type (enter ? to see all types) [string]:
>

Field length [255]:
> 32

Can this field be null in the database (nullable) (yes/no) [no]:
>

updated: src/Entity/membre.php

Add another property? Enter the property name (or press <return> to stop adding fields):
> prenomMembre

Field type (enter ? to see all types) [string]:
>

Field length [255]:
> 20

Can this field be null in the database (nullable) (yes/no) [no]:
>

updated: src/Entity/Membre.php

Add another property? Enter the property name (or press <return> to stop adding fields):
> telMembre

Field type (enter ? to see all types) [string]:
>

Field length [255]:
> 14

Can this field be null in the database (nullable) (yes/no) [no]:
> yes

updated: src/Entity/membre.php

Add another property? Enter the property name (or press <return> to stop adding fields):
>

… et      mailMembre, rueMembre, cpMembre, villeMembre

 Success!


Next: When you're ready, create a migration with make:migration
```

4. Créer la table membre

Grâce au DoctrineMigrationsBundle il est possible de créer la table à partir de l'entité.
Pour cela, il faut générer la migration et ensuite l'exécuter.

Générez la migration :

Saisir la commande **php bin/console make:migration** pour créer la table membre à partir de l'entité qui a été précédemment générée.

```
C:\wamp64\www\agoram61>php bin/console make:migration
```

```
C:\Windows\System32\cmd.e
C:\wamp64\www\agoram61>php bin/console make:migration

 created: migrations/Version20251105194559.php

 Success!

 Review the new migration then run it with php bin/console doctrine:migrations:migrate
 See https://symfony.com/doc/current/bundles/DoctrineMigrationsBundle/index.html

C:\wamp64\www\agoram61>
```

Consulter le fichier généré migrations/Version20251105194559.php qui contient les ordres SQL qui seront exécutés lorsque vous exécuterez ensuite la migration.

Fichier agoram61/migrations/Version20251105194559

```php
<?php

declare(strict_types=1);

namespace DoctrineMigrations;

use Doctrine\DBAL\Schema\Schema;
use Doctrine\Migrations\AbstractMigration;

/**
 * Auto-generated Migration: Please modify to your needs!
 */
final class Version20251105194559 extends AbstractMigration
{
    public function getDescription(): string
    {
        return '';
    }

    public function up(Schema $schema): void
    {
        // this up() migration is auto-generated, please modify it to your needs
        $this->addSql('CREATE TABLE membre (id INT AUTO_INCREMENT NOT NULL,
username VARCHAR(180) NOT NULL, roles JSON NOT NULL, password VARCHAR(255) NOT
NULL, nom_membre VARCHAR(32) NOT NULL, prenom_membre VARCHAR(20) NOT NULL,
tel_membre VARCHAR(14) DEFAULT NULL, mail_membre VARCHAR(100) NOT NULL, rue_membre
VARCHAR(255) DEFAULT NULL, cp_membre VARCHAR(10) DEFAULT NULL, ville_membre
VARCHAR(100) DEFAULT NULL, UNIQUE INDEX UNIQ_F6B4FB29F85E0677 (username), PRIMARY
KEY(id)) DEFAULT CHARACTER SET utf8mb4 COLLATE `utf8mb4_unicode_ci` ENGINE =
InnoDB');
    }

    public function down(Schema $schema): void
    {
        // this down() migration is auto-generated, please modify it to your needs
```

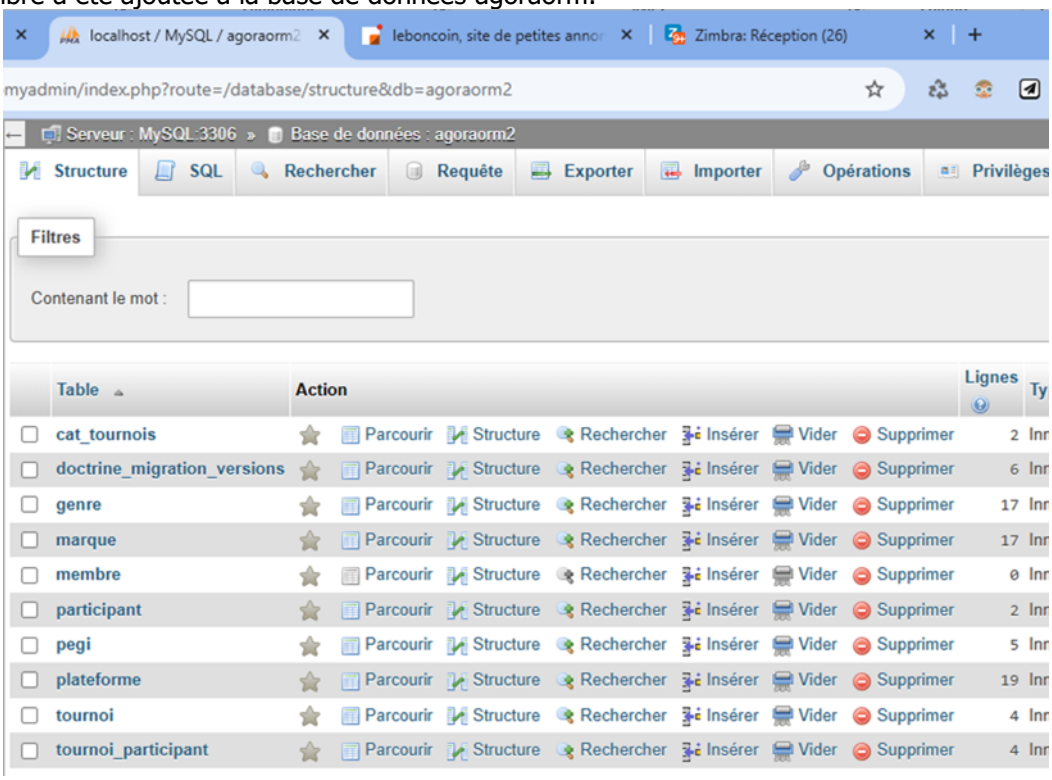
```
        $this->addSql('DROP TABLE membre');
    }
}
```

Exécutez la migration :

```
C:\wamp64\www\agoram61>php bin/console doctrine:migrations:migrate
```

```
C:\Windows\System32\cmd.e
C:\wamp64\www\agoram61>php bin/console doctrine:migrations:migrate

 WARNING! You are about to execute a migration in database "agoraorm2" that could result in schema changes and data loss
. Are you sure you wish to continue? (yes/no) [yes]:
 >

[notice] Migrating up to DoctrineMigrations\Version20251105194559
[notice] finished in 138.1ms, used 20M memory, 1 migrations executed, 1 sql queries


C:\wamp64\www\agoram61>
```

La table membre a été ajoutée à la base de données agoraorm.



Remarques :

- Pour ajouter des champs par la suite, il est possible de repasser les 3 commandes :
  php bin/console make:entity on ne saisit que les nouvelles propriétés
  php bin/console make:migration un ALTER TABLE est généré
  php bin/console doctrine:migrations:migrate exécuter les ordres pour la migration

- si vous ajoutez des propriétés directement dans le code, vous pouvez générer les getters et setters avec :
  php bin/console make:entity --regenerate

2.3 Créer des utilisateurs de test avec DoctrineFixturesBundle

Il existe plusieurs façons de créer des données de tests.

Doctrine fournit une bibliothèque (plus précisément un bundle) nommé **DoctrineFixturesBundle** qui facilite la mise en place des données de tests ou **fixtures**.

Les ***fixtures*** ou données de tests permettent au développeur d'initialiser les données de la base de données de test dans un état connu. Par exemple, pour l'entité membre, il est possible de créer plusieurs membres avec des caractéristiques différentes. **Les fixtures** permettent donc d'utiliser un environnement de développement avec de fausses données afin de se rapprocher d'un environnement de production.

Dans bien des cas, il faudra recharger les fixtures entre chaque test. De cette manière, les effets de bord seront évités. Un état stable et connu sera initialisé entre chaque test.

1. Installez le bundle DoctrineFixturesBundle

Installer le composant orm-fixtures :

```
C:\wamp64\www\agoram61>composer require --dev orm-fixtures
```

```
C:\Windows\System32\cmd.e

C:\wamp64\www\agoram61>composer require --dev orm-fixtures
./composer.json has been updated
Running composer update doctrine/doctrine-fixtures-bundle
Loading composer repositories with package information
Updating dependencies
Lock file operations: 2 installs, 0 updates, 0 removals
  - Locking doctrine/data-fixtures (1.8.2)
  - Locking doctrine/doctrine-fixtures-bundle (3.6.2)
Writing lock file
Installing dependencies from lock file (including require-dev)
Package operations: 2 installs, 0 updates, 0 removals
  - Downloading doctrine/data-fixtures (1.8.2)
  - Downloading doctrine/doctrine-fixtures-bundle (3.6.2)
  - Installing doctrine/data-fixtures (1.8.2): Extracting archive
  - Installing doctrine/doctrine-fixtures-bundle (3.6.2): Extracting archive
Package doctrine/cache is abandoned, you should avoid using it. No replacement was suggested.
Package sensio/framework-extra-bundle is abandoned, you should avoid using it. Use Symfony instead.
Generating optimized autoload files
Class PdoJeux located in ./src/Controller/modele/class.PdoJeux.inc.php does not comply with psr-4 autoloading standard (
rule: App\ => ./src). Skipping.
69 packages you are using are looking for funding.
Use the `composer fund` command to find out more!

Symfony operations: 1 recipe (c3ea31b6503434eeb00afa1afad76234)
  - Configuring doctrine/doctrine-fixtures-bundle (>=3.0): From github.com/symfony/recipes:main
Executing script cache:clear [OK]
Executing script assets:install public [OK]

 What's next?


Some files have been created and/or updated to configure your new packages.
Please review, edit and commit them: these files are yours.

Found 10 security vulnerability advisories affecting 7 packages.
Run "composer audit" for a full list of advisories.
Using version ^3.6 for doctrine/doctrine-fixtures-bundle
```

option --dev : Les fixtures sont uniquement dédiées à des environnements de développement, c'est pourquoi on précise que la configuration de ce bundle est uniquement liée à cet environnement.

Vous pouvez voir cette configuration dans le fichier agora5/config/**bundles.php** :

config > bundles.php

```php
<?php

return [
    Symfony\Bundle\FrameworkBundle\FrameworkBundle::class => ['all' => true],
    Doctrine\Bundle\DoctrineBundle\DoctrineBundle::class => ['all' => true],
    Doctrine\Bundle\MigrationsBundle\DoctrineMigrationsBundle::class => ['all' => true],
    Symfony\Bundle\DebugBundle\DebugBundle::class => ['dev' => true],
    Symfony\Bundle\TwigBundle\TwigBundle::class => ['all' => true],
    Symfony\Bundle\WebProfilerBundle\WebProfilerBundle::class => ['dev' => true, 'test' => true],
    Twig\Extra\TwigExtraBundle\TwigExtraBundle::class => ['all' => true],
    Symfony\Bundle\SecurityBundle\SecurityBundle::class => ['all' => true],
    Symfony\Bundle\MonologBundle\MonologBundle::class => ['all' => true],
    Symfony\Bundle\MakerBundle\MakerBundle::class => ['dev' => true],
    Doctrine\Bundle\FixturesBundle\DoctrineFixturesBundle::class => ['dev' => true, 'test' => true],
];
```

2. Une nouvelle classe a été créée dans DataFixtures

Fichier agoram61/src/DataFixtures/AppFixtures.php

```php
<?php

namespace App\DataFixtures;

use Doctrine\Bundle\FixturesBundle\Fixture;
use Doctrine\Persistence\ObjectManager;

class AppFixtures extends Fixture
{
    public function load(ObjectManager $manager): void
    {
        // $product = new Product();
        // $manager->persist($product);

        $manager->flush();
    }
}
```

3. Nous allons créer une classe spécifique pour les données de test des membres à l'aide de la commande **make:fixtures** pour créer la classe MembreFixtures.
C'est dans cette classe que nous ajouterons le code pour créer des utilisateurs de test :

```
C:\wamp64\www\agoram61>php bin/console make:fixtures
```

```
C:\Windows\System32\cmd.e
C:\wamp64\www\agoram61>php bin/console make:fixtures

 The class name of the fixtures to create (e.g. AppFixtures):
 > MembreFixtures

 created: src/DataFixtures/MembreFixtures.php

 Success!

 Next: Open your new fixtures class and start customizing it.
 Load your fixtures by running: php bin/console doctrine:fixtures:load
 Docs: https://symfony.com/doc/current/bundles/DoctrineFixturesBundle/index.html
```

La classe MembreFixtures se trouve dans le fichier agora5/src/DataFixtures/MembreFixtures.php

4. Certaines bibliothèques permettent de générer des données qui semble réelles et cohérentes contrairement à des valeurs aléatoires type "Lorem ipsum".
C'est le cas de **Faker** qui est une bibliothèque PHP. Cette bibliothèque permet de générer facilement tout type de donnée. Malheureusement, cette bibliothèque recroît de moins en moins de versions de mise à jour.

Plus d'informations ici : https://fakerphp.github.io/
Liste des champs qui peuvent être générés : https://v6.fakerjs.dev/api/unique.html

Commençons par ajouter la bibliothèque Faker au projet :

```
C:\wamp64\www\agoram61>composer require --dev fakerphp/faker
```

```
C:\Windows\System32\cmd.e

C:\wamp64\www\agoram61>composer require --dev fakerphp/faker
./composer.json has been updated
Running composer update fakerphp/faker
Loading composer repositories with package information
Updating dependencies
Lock file operations: 1 install, 0 updates, 0 removals
  - Locking fakerphp/faker (v1.24.1)
Writing lock file
Installing dependencies from lock file (including require-dev)
Package operations: 1 install, 0 updates, 0 removals
  - Downloading fakerphp/faker (v1.24.1)
  - Installing fakerphp/faker (v1.24.1): Extracting archive
Package doctrine/cache is abandoned, you should avoid using it. No replacement was suggested.
Package sensio/framework-extra-bundle is abandoned, you should avoid using it. Use Symfony instead.
Generating optimized autoload files
Class PdoJeux located in ./src/Controller/modele/class.PdoJeux.inc.php does not comply with psr-4 autoloading standard (
rule: App\ => ./src). Skipping.
69 packages you are using are looking for funding.
Use the `composer fund` command to find out more!

Run composer recipes at any time to see the status of your Symfony recipes.

Executing script cache:clear [OK]
Executing script assets:install public [OK]

Found 10 security vulnerability advisories affecting 7 packages.
Run "composer audit" for a full list of advisories.
Using version ^1.24 for fakerphp/faker
```

5. Complétez la classe MembreFixtures pour créer 10 utilisateurs.
Recopiez tout le code dans votre environnement

Fichier agoram61/src/DataFixtures/MembreFixtures.php

```php
<?php
namespace App\DataFixtures;

use Doctrine\Bundle\FixturesBundle\Fixture;
use Doctrine\Persistence\ObjectManager;

use Faker\Factory;
use Faker\Generator;

use App\Entity\Membre;
use Symfony\Component\PasswordHasher\Hasher\UserPasswordHasherInterface;

class MembreFixtures extends Fixture
{
    private Generator $faker;
    private UserPasswordHasherInterface $passwordHasher;
```

```php
    public function __construct(UserPasswordHasherInterface $unPasswordHasher)
    {
        $this->passwordHasher = $unPasswordHasher;
        $this->faker = Factory::create('fr_FR');
    }

    public function load(ObjectManager $manager): void
    {
         // créer 10 membres
         for ($i = 0; $i < 10; $i++) {
            $membre = new Membre();
            $membre->setUsername($this->faker->lastName);
            $membre->setNomMembre($this->faker->lastName);
            $membre->setPrenomMembre($this->faker->firstName);
            $membre->setTelMembre(substr($this->faker->e164PhoneNumber, 2, 10));
            $membre->setMailMembre(sprintf('userdemo%d@exemple.com', $i));
            $membre->setPassword($this->passwordHasher->hashPassword($membre, 'userdemo'));
            $membre->setRueMembre($this->faker->streetAddress);
            $membre->setVilleMembre($this->faker->city);
            $membre->setCpMembre($this->faker->numberBetween(111111, 999999));
            $manager->persist($membre);         }
        $manager->flush();
    }
}
```

6. Lancez la génération des fixtures, ici **on lance seulement les MembreFixtures** (--group=MembreFixtures) et on ajoute les données à la base de données existante (--append).
Par défaut la commande load supprime toutes les données de la base (purge) et pour l'éviter, il faut indiquer l'option --append

```
C:\wamp64\www\agoram61>php bin/console doctrine:fixtures:load --group=MembreFixtures --append
```

```
C:\Windows\System32\cmd.e
C:\wamp64\www\agoram61>php bin/console doctrine:fixtures:load --group=MembreFixtures --append

   > loading App\DataFixtures\MembreFixtures

C:\wamp64\www\agoram61>
```

Aide sur la commande load : `php bin/console doctrine:fixtures:load --help`
Documentation ici : https://symfony.com/bundles/DoctrineFixturesBundle/current/index.html

Si vous obtenez le message d'erreur suivant, c'est que la génération des fixtures a généré des données identiques en particulier sur les champs qui doivent être unique (dans le cas suivant deux membres avec le même nom). Relancer simplement la commande en espérant que cette fois ci les 10 jeux de données seront uniques.

```
C:\Windows\System32\cmd.e
C:\wamp64\www\agoram61>php bin/console doctrine:fixtures:load --group=MembreFixtures --append

   > loading App\DataFixtures\MembreFixtures

In ExceptionConverter.php line 64:

  An exception occurred while executing a query: SQLSTATE[23000]: Integrity constraint violation: 1062 Duplicata
  du champ 'Marechal' pour la clef 'membre.UNIQ_F6B4FB29F85E0677'

In Exception.php line 24:

  SQLSTATE[23000]: Integrity constraint violation: 1062 Duplicata du champ 'Marechal' pour la clef 'membre.UNIQ_F
  6B4FB29F85E0677'

In Statement.php line 130:

  SQLSTATE[23000]: Integrity constraint violation: 1062 Duplicata du champ 'Marechal' pour la clef 'membre.UNIQ_F
  6B4FB29F85E0677'

doctrine:fixtures:load [--append] [--group GROUP] [--em EM] [--purger PURGER] [--purge-exclusions PURGE-EXCLUSIONS]
[--purge-with-truncate]
```

C:\wamp64\www\agoram61>php bin/console doctrine:fixtures:load --group=MembreFixtures --append

10 utilisateurs ont été ajoutés à la table membre :

id	username	roles	password	nom_membre	prenom_membre	mail_membre	tel_me
50	Brunel	[]	$2y$13$woLxsKHMbI40pkZEEfcfQuVmKO4HPTgEPBygUWDU7Kg...	Menard	Aimé	userdemo9@exemple.com	39861(
45	Charles	[]	$2y$13$0avENU6xoGKTtb8CbIfz8OV2BzS7RM/XiduMNICYnWB...	Mary	Antoine	userdemo4@exemple.com	371905
46	Delmas	[]	$2y$13$qqX7ObgT2Vu98VGQkcdrFO9YeEpC8.mqDbj7aZKODwf...	Gillet	Brigitte	userdemo5@exemple.com	306007
49	Courtois	[]	$2y$13$kLuG8Y.dxwwHC2Ss5D7XmeMlZF8yqt/TUr5Au98diz1...	Dias	Gilles	userdemo8@exemple.com	37330
48	Pascal	[]	$2y$13$xPk6usBrwPiwewGO0J8m7u2KW.47PalJUXioTfnznJA...	Teixeira	Charles	userdemo7@exemple.com	367805
47	Roger	[]	$2y$13$HSYF0sU9k/xKfdelKfmNN.xb5yYWqSoqKrylA4QjI6i...	Roy	Dominique	userdemo6@exemple.com	34167(
44	Pottier	[]	$2y$13$D.SZxOeG5QcJH9wMxOCXmOtQKL75VioilByC03mykBm...	Mathieu	Arthur	userdemo3@exemple.com	382379
43	Blin	[]	$2y$13$5JlL3CK7EEEifiAX8XWJ4.9zew/.GN0nb5lJmVCeAQ1...	Pages	Antoine	userdemo2@exemple.com	337645
42	Sauvage	[]	$2y$13$91nzBfY5E1e5ynX6W3qKDO0glwNyQjVV1lNFDqA.4iV...	Langlois	Océane	userdemo1@exemple.com	318267
41	Jacques	[]	$2y$13$jPthORRrVnAF7gRXK2.L/.35s8hqATe7XkC4Tr6F2Bq...	Robin	Arnaude	userdemo0@exemple.com	382474

A SAVOIR (inutile de la faire ici)

Pour vider la base de données et charger les fixtures (données de tests) il est possible de passer la commande :

```
C:\wamp64\www\agora5>php bin/console doctrine:fixtures:load

Careful, database "agoraorm" will be purged. Do you want to continue? (yes/no) [no]:
> yes

  > purging database
  > loading App\DataFixtures\AppFixtures
  > loading App\DataFixtures\ProduitFixture

C:\wamp64\www\agora5>
```

2.4 Gérer l'authentification

Rappel : S'authentifier, c'est confirmer son identité en fournissant par exemple un nom d'utilisateur (username) ou un email et un mot de passe sur un système.

2.4.1 Créer un formulaire de connexion

1. Pour créer ce formulaire et toute la logique d'authentification, nous utiliserons la commande ***make:auth***

```
C:\wamp64\www\agoram61>php bin/console make:auth
```

```
C:\Windows\System32\cmd.e
C:\wamp64\www\agoram61>php bin/console make:auth

 What style of authentication d
  [0] Empty authenticator
  [1] Login form authenticator
 > 1

 The class name of the auth  to create (e.g. AppCustomAuthenticator):
 > LoginForm

 Choose a name for the controller class (e.g. SecurityControll
 >

 Do you want to generat  URL? (yes/no) [yes]:
 > yes

 Do you want to support remember me? (yes/no) [yes]:
 > no

 created: src/Security/LoginFormAuthenticator.php
 updated: config/packages/security.yaml
 created: src/Controller/SecurityController.php
 created: templates/security/login.html.twig

 Success!

 Next:
 - Customize your new authenticator.
 - Finish the redirect "TODO" in the App\Security\LoginFormAuthenticator::onAuthenticationSuccess() method.
 - Review & adapt the login template: templates/security/login.html.twig.
```

Le nom de la classe qui servira pour l'authentification

nous souhaitons une route pour la déconnexion, choisir *yes*

Cette commande va générer **3 nouveaux fichiers** :

- ***LoginFormAuthenticator.php*** dans src/Security/ : cette classe expose les méthodes pour récupérer l'utilisateur dans la base de données, vérifier le mot de passe, etc ...
- ***SecurityController.php*** dans src/Controller/ : les contrôleurs pour la connexion login() et la déconnexion logout() y sont définis.
- ***login.html.twig*** dans templates/security/ : la vue qui affiche le formulaire de connexion.

La commande a également modifié le fichier ***security.yaml*** qui se trouve dans config/packages/ et y a ajouté LognForm comme authentificator et une entrée pour le logout. En jaune les modifications :

Fichier agoram61/config/packages/security.yml

```yaml
security:
    # https://symfony.com/doc/current/security.html#registering-the-user-hashing-passwords
    password_hashers:
        Symfony\Component\Security\Core\User\PasswordAuthenticatedUserInterface: 'auto'
    # https://symfony.com/doc/current/security.html#loading-the-user-the-user-provider
    providers:
        # used to reload user from session & other features (e.g. switch_user)
        app_user_provider:
            entity:
```

```
                class: App\Entity\Membre
                property: username
    firewalls:
        dev:
            pattern: ^/(_(profiler|wdt)|css|images|js)/
            security: false
        main:
            lazy: true
            provider: app_user_provider
            custom_authenticator: App\Security\LoginFormAuthenticator
            logout:
                path: app_logout
                # where to redirect after logout
                # target: app_any_route

            # activate different ways to authenticate
            # https://symfony.com/doc/current/security.html#the-firewall

            # https://symfony.com/doc/current/security/impersonating_user.html
            # switch_user: true

    # Easy way to control access for large sections of your site
    # Note: Only the *first* access control that matches will be used
    access_control:
        # - { path: ^/admin, roles: ROLE_ADMIN }
        # - { path: ^/profile, roles: ROLE_USER }

when@test:
    security:
        password_hashers:
            # By default, password hashers are resource intensive and take time. This is
            # important to generate secure password hashes. In tests however, secure hashes
            # are not important, waste resources and increase test times. The following
            # reduces the work factor to the lowest possible values.
            Symfony\Component\Security\Core\User\PasswordAuthenticatedUserInterface:
                algorithm: auto
                cost: 4 # Lowest possible value for bcrypt
                time_cost: 3 # Lowest possible value for argon
                memory_cost: 10 # Lowest possible value for argon
```

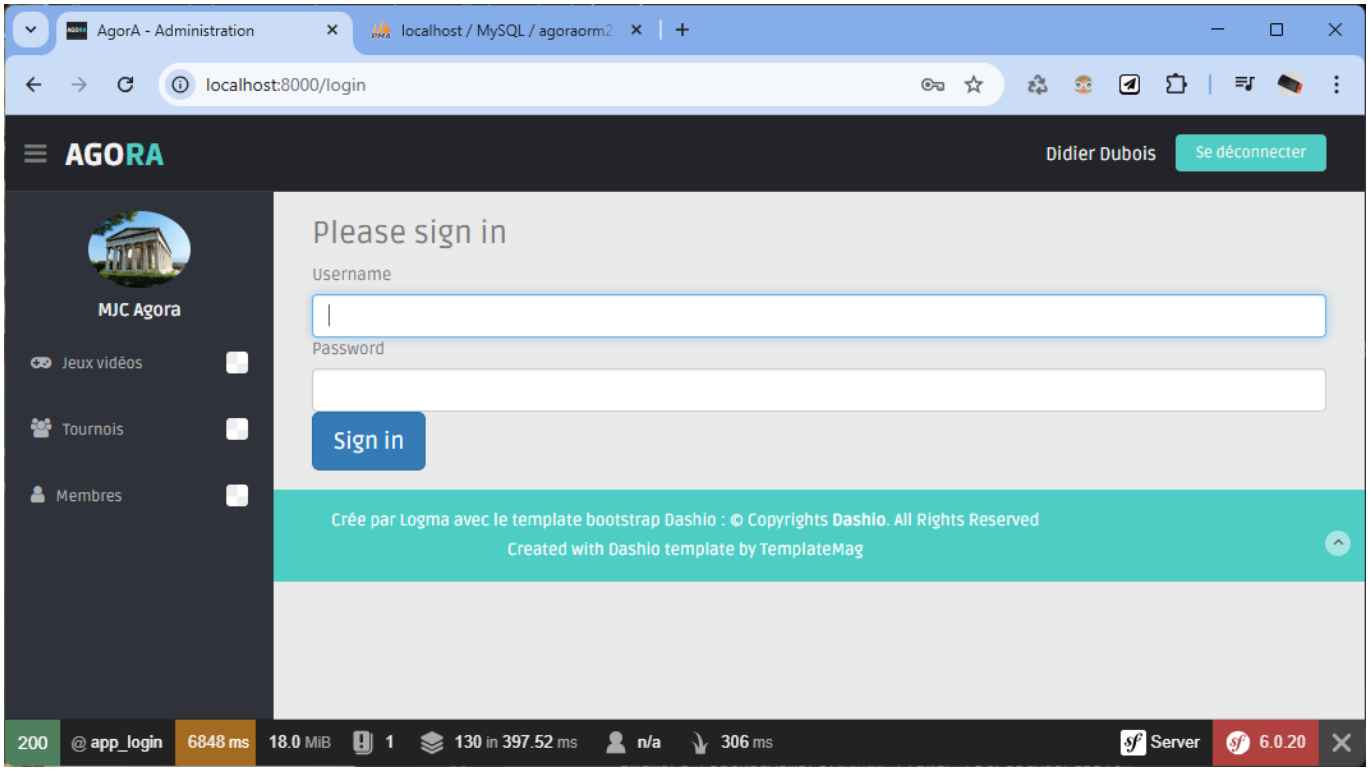
indiquons ici que le formulaire à utiliser est celui créé précédemment

ici le nom logique de la route à utiliser pour la deconnexion

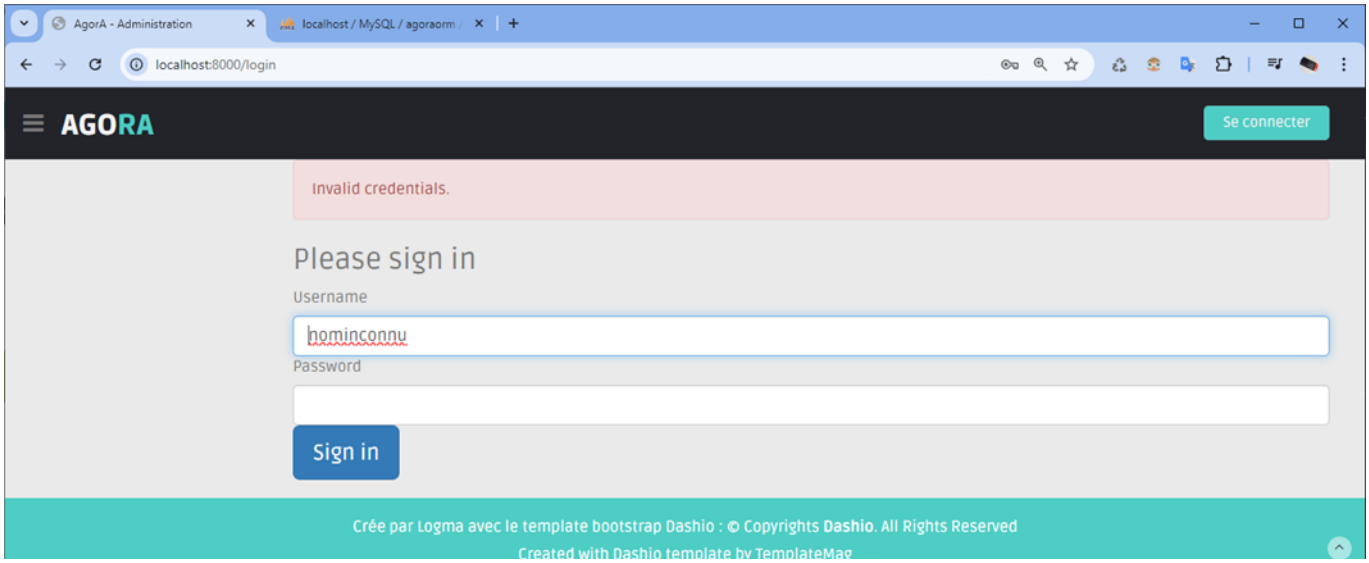
2. Démarrer le serveur Symfony et testez avec http://127.0.0.1:8000/login

Pour rappel, un serveur web est intégré à Symfony depuis la version 5.4 de PHP. Ainsi on est sûr que le serveur utilisera la même version de PHP que celle utilisée en ligne de commande. Pour le démarrer, lancez la commande : **symfony server:start** **(ou symfony serve)**

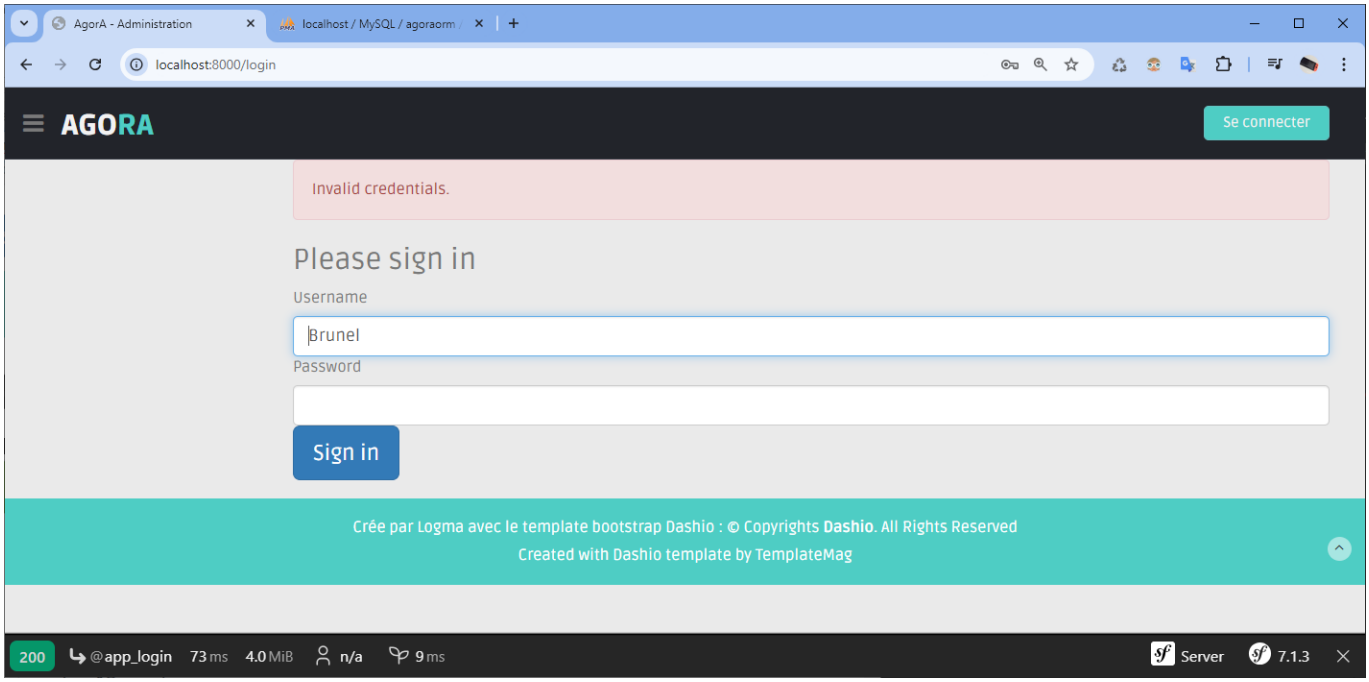
Appeler la route http://127.0.0.1:8000/login



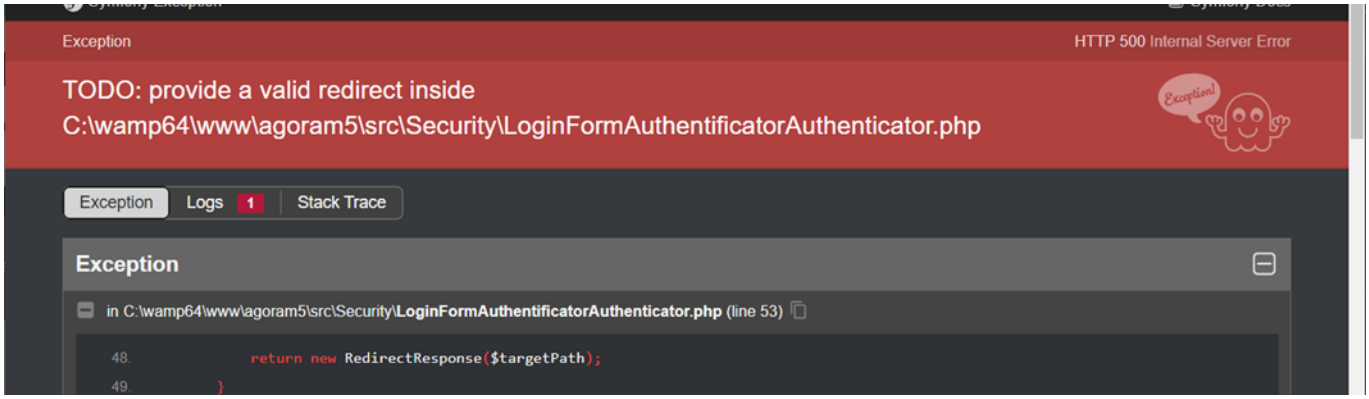
Si vous saisissez username inconnu en base de données, vous obtenez :



Si vous saisissez un username existant en base de données, mais un mot de passe incorrect, vous obtenez :



Si vous saisissez des identifiants (credentials) corrects (username de la base de données avec mot de passe userdemo), vous obtenez l'erreur suivante :



En effet, l'utilisateur authentifié doit être redirigé vers la page d'accueil. Pour cela il faut modifier la fonction onAuthenticationSuccess() .

3. Rediriger l'utilisateur authentifié vers la page d'accueil :

Fichier agoram61/src/Security/LoginFormAuthenticator.php

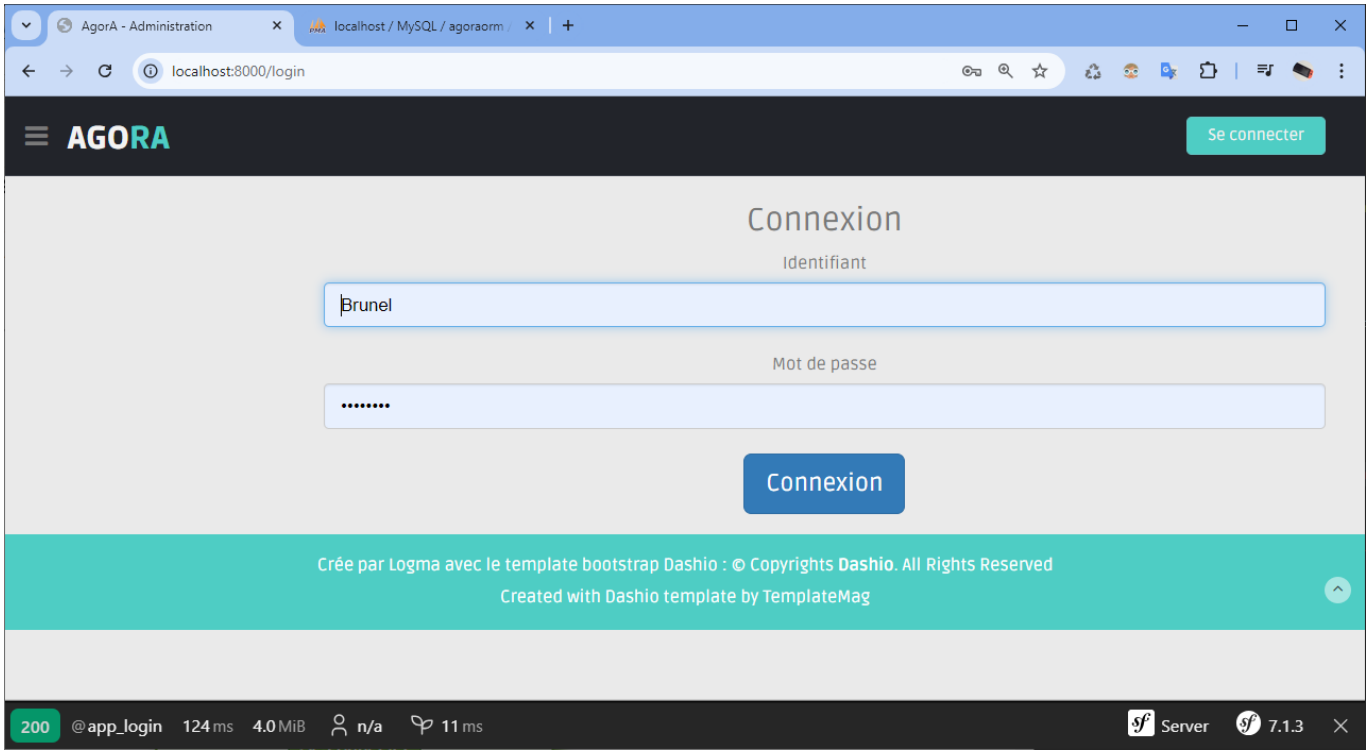
```
. . .
    public function onAuthenticationSuccess(Request $request, TokenInterface
$token, string $firewallName): ?Response
    {
        if ($targetPath = $this->getTargetPath($request->getSession(), $firewallName)) {
            return new RedirectResponse($targetPath);
        }
        return new RedirectResponse($this->urlGenerator->generate('accueil'));
    }

. . .
```

4. Franciser et adapter le design de la page de connexion :

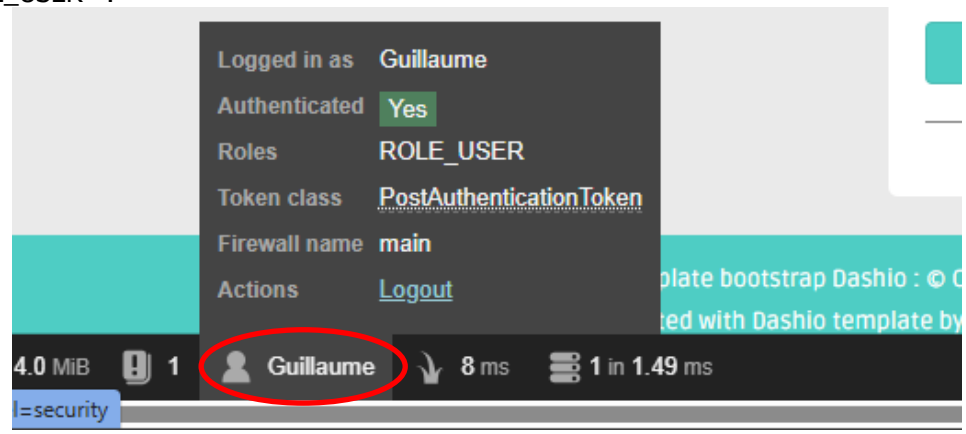
Fichier agoram61/templates/security/login.html.twig

```
{% extends 'base.html.twig' %}

{% block title %}Se connecter{% endblock %}

{% block body %}
<div align="center">
    <form method="post">
        {% if error %}
            <div class="alert alert-danger">{{ error.messageKey|trans(error.messageData, 'security') }}</div>
        {% endif %}

        {% if app.user %}
            <div class="mb-3">
                Vous êtes connecté en tant que  {{ app.user.userIdentifier }}, <a
href="{{ path('app_logout') }}">Déconnecter</a>
            </div>
        {% endif %}

        <h1 class="h3 mb-3 font-weight-normal">Connexion</h1>
        <label for="inputEmail">Identifiant</label>
       <input type="text" value="{{ last_username }}" name="username"
id="inputUsername" class="form-control" autocomplete="username" required
autofocus>
        <label for="inputPassword">Mot de passe</label>
        <input type="password" name="password" id="inputPassword" class="form-
control" autocomplete="current-password" required>

        <input type="hidden" name="_csrf_token"
            value="{{ csrf_token('authenticate') }}"
        >

        <button class="btn btn-lg btn-primary" type="submit">
            Se connecter
        </button>
    </form>
</div>
{% endblock %}
```

5. Tester la connexion avec le nouveau formulaire francisé et charté :



Après connexion, la debugbar en bas de page permet de voir avec quel membre vous êtes authentifié.
On obtient une série d'information sur le membre connecté dont son rôle, sachant que pour le moment, ils ont tous pour role « ROLE_USER ».



2.4.1 Afficher le nom de l'utilisateur

Commençons par afficher le nom de l'utilisateur dans la barre de navigation, veillons à afficher « se déconnecter » quand personne n'est connecté.

Pour récupérer l'utilisateur connecté dans une vue twig on écrit *{{ app.user }}*, et on peut accéder à tous ses attributs comme son nom avec *{{ app.user.nomMembre }}*.
Dans un contrôleur qui hérite de la classe *AbstractController* de Symfony, pour récupérer l'utilisateur connecté on écrit *$this->getUser()*. Là aussi vous pouvez récupérer son nom avec *$this->getUser()->getNomMembre()*.

Pour cela il faut modifier le template de base *base.html.twig* où se trouve la barre de navigation:

Fichier agoram61/templates/base.html.twig

```
...
<body>
   <section id="container">
   <!--
   ************************************************************************
*****************
```

```
   TOP BAR CONTENT & NOTIFICATIONS
   ******************************************************************************
*****************
   -->
   <!--header start-->
   <header class="header black-bg">
   <div class="sidebar-toggle-box">
      <div class="fa fa-bars tooltips" data-placement="right" data-original-title="Toggle Navigation"></div>
   </div>
   <!--logo start-->
   <a href="{{path('accueil')}}" class="logo"><b>Ago<span>rA</span></b></a>
   <!--logo end-->
   <div class="top-menu">
      <ul class="nav pull-right top-menu">
      {# app.session.has() nouvelle syntaxe pour tester si une variable de session existe #}
      {# app.session.get() nouvelle syntaxe pour récupérer la valeur d'une variable de session #}
      {% if (app.user) %}
         <a href="" class="userAgo">{{ app.user.prenomMembre }} {{ app.user.nomMembre }}</a>
         <li><a class="logout" href="{{ path('app_logout') }}">Déconnecter</a></li>
      {% else %}
         <li><a class="logout" href="{{ path('app_login') }}">Se connecter</a></li>
      {% endif %}
      </ul>
   </div>
   </header>
   <!--header end-->
   <!--
   ******************************************************************************
*****************
   MAIN SIDEBAR MENU
   ******************************************************************************
*****************
   -->
   {% block menu %}
      {% if (app.user) %}
         {{ include('menu.html.twig') }}
      {% endif %}
   {% endblock %}
   <!--
...
```

Modifier le template **accueil.html.twig** pour afficher le nom et le prénom de l'utilisateur connecté sur la page d'accueil :

Fichier agoram61/templates/accueil.html.twig

```
{% extends "base.html.twig" %}
{% block central %}
<!--
******************************************************************************
******************
 MAIN CONTENT
```

```
*****************************************************************************
*****************
-->
<div class="container">
    <div id="blocImage">
        <img src="assets/img/greece-1162816_640.jpg" alt="Agora LR" class="responsive">
        {# <img src="assets/img/greece-3348294_1280.jpg" alt="Agora HR" class="responsive"> #}
        <div id="blocImageTexte">
            {% if app.user %}
            Bonjour {{ app.user.prenomMembre }} {{ app.user.nomMembre }}
            {% else %}
            Identifiez vous !
            {% endif %}
        </div>
    </div>
</div>
{% endblock %}
```

Modifier le contrôleur **AccueilControleur.php** pour basculer sur le formulaire de connexion ou sur la page d'accueil si on un utilisateur est identifié ou pas

Fichier agoram61/src/Controller/AccueilControler.php

```
<?php
// src/Controller/AccueilController.php
namespace App\Controller;

require_once 'modele/class.PdoJeux.inc.php';

//use Symfony\Component\HttpFoundation\Response;
use Symfony\Component\Routing\Attribute\Route;
use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
use Symfony\Component\HttpFoundation\Session\SessionInterface;

class AccueilController extends AbstractController
{
    #[Route('/', name: 'accueil')]
    public function index(SessionInterface $session)
    {
        // si un utilisateur est connecté on affiche la page d'accueil
        if ($this->getUser()) {
            return $this->render('accueil.html.twig');
        } else {
            // sinon on affiche la page de connexion
            return $this->render('security/login.html.twig');
        }
    }
}
```

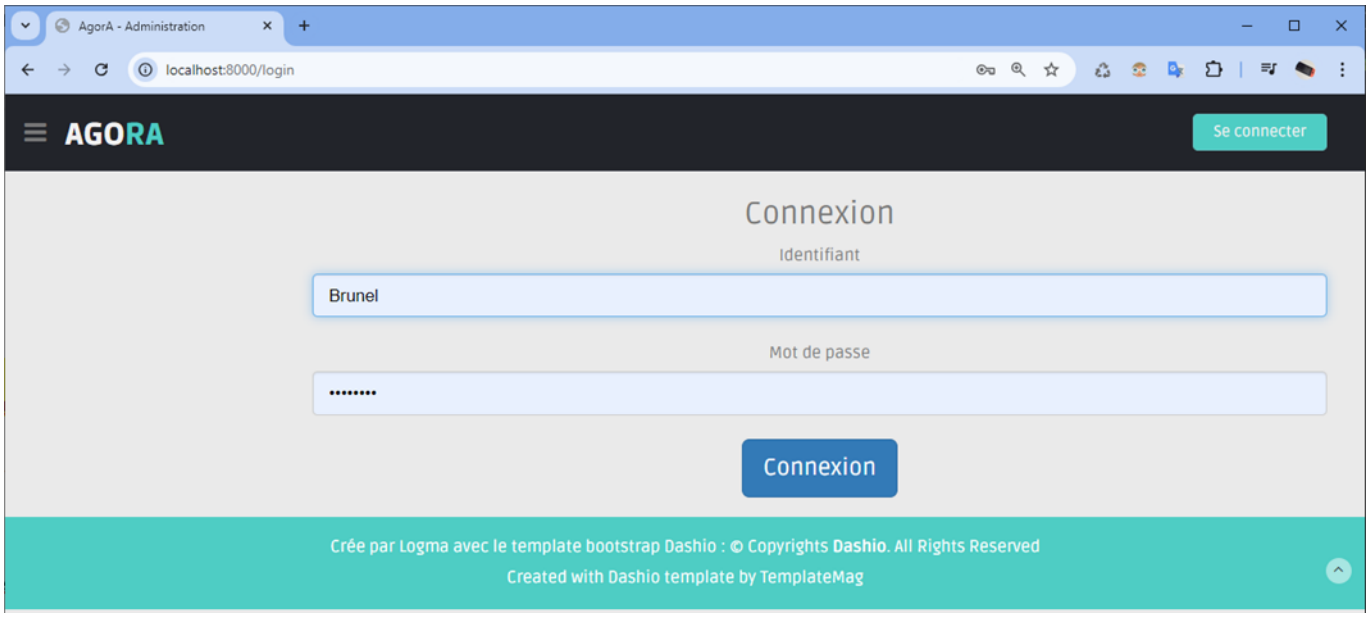
Modifier le formulaire de connexion **login.html.twig** :

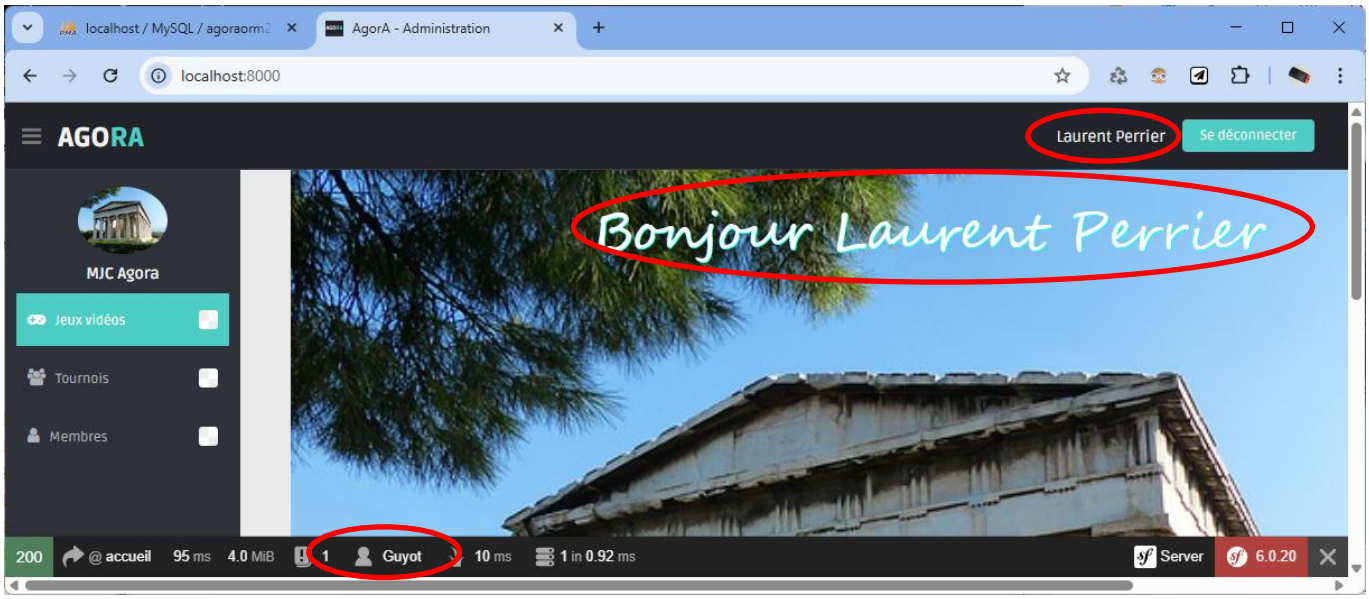
Fichier agoram61/templates/security/login.html.twig

```
{% extends 'base.html.twig' %}

{% block title %}Connexion{% endblock %}
```

```
{% block central %}
{# <div class="container connexion"> #}
<div align="center">
<form method="post" action="{{ path('app_login') }}">
    {% if error is defined %}
        {% if error %}
        <div class="alert alert-danger">{{
error.messageKey|trans(error.messageData, 'security') }}</div>
        {% endif %}
    {% endif %}

    {% if app.user %}
        <div class="mb-3">
            Vous êtes connecté en tant que {{ app.user.userIdentifier }}, <a
href="{{ path('app_logout') }}">Déconnecter</a>
        </div>
    {% endif %}
```

Authentifiez-vous avec un username et un mot de passe valide, vérifiez que la connexion se passe bien et que vous accédez bien à l'accueil de l'application.





2.4.2 Gérer la déconnexion

Pour revenir à la page de connexion lorsqu'on se déconnecte, il suffit d'indiquer la route du login dans **security.yaml**

Fichier agora5/config/packages/security.yaml

```
security:
    # https://symfony.com/doc/current/security.html#registering-the-user-hashing-passwords
    password_hashers:
        Symfony\Component\Security\Core\User\PasswordAuthenticatedUserInterface: 'auto'
    # https://symfony.com/doc/current/security.html#loading-the-user-the-user-provider
    providers:
        # used to reload user from session & other features (e.g. switch_user)
        app_user_provider:
            entity:
                class: App\Entity\Membre
                property: username
    firewalls:
        dev:
            pattern: ^/(_(profiler|wdt)|css|images|js)/
            security: false
        main:
            lazy: true
            provider: app_user_provider
            custom_authenticator: App\Security\LoginFormAuthenticatorAuthenticator
            logout:
                path: app_logout
                # where to redirect after logout
                target: app_login

            # activate different ways to authenticate
            # https://symfony.com/doc/current/security.html#the-firewall

            # https://symfony.com/doc/current/security/impersonating_user.html
```

```
            # switch_user: true

    # Easy way to control access for large sections of your site
    # Note: Only the *first* access control that matches will be used
    access_control:
        # - { path: ^/admin, roles: ROLE_ADMIN }
        # - { path: ^/profile, roles: ROLE_USER }

when@test:
    security:
        password_hashers:
            # By default, password hashers are resource intensive and take time.
This is
            # important to generate secure password hashes. In tests however,
secure hashes
            # are not important, waste resources and increase test times. The
following
            # reduces the work factor to the lowest possible values.
            Symfony\Component\Security\Core\User\PasswordAuthenticatedUserInterfac
e:
                algorithm: auto
                cost: 4 # Lowest possible value for bcrypt
                time_cost: 3 # Lowest possible value for argon
                memory_cost: 10 # Lowest possible value for argon
```

2.5 Gérer les membres

Il est possible de générer un formulaire d'inscription (avec make:registration-form), mais nous n'utiliserons pas cette possibilité ici, puisque, dans le contexte de l'association Agora, ce n'est pas l'utilisateur qui peut s''inscrire mais l'administrateur qui inscrira les utilisateurs.
Seul le rôle ROLE_ADMIN aura accès à la gestion des utilisateurs.

Pour gérer les CRUD membres nous allons utiliser ici un autre générateur du MakerBundle, **make:crud**
Quand vous aurez appris à gérer vous-mêmes les CRUD, vous serez à même de comprendre le code généré.
Et bien sûr, le make:crud peut être utilisé avec toute entité et participe donc à la productivité du développeur.
Documentation ici : https://symfony.com/blog/new-and-improved-generators-for-makerbundle

1. La commande **make:crud** permet de générer une interface CRUD de base à partir de l'entité donnée.

```
C:\wamp64\www\agoram61>php bin/console make:crud Membre
```

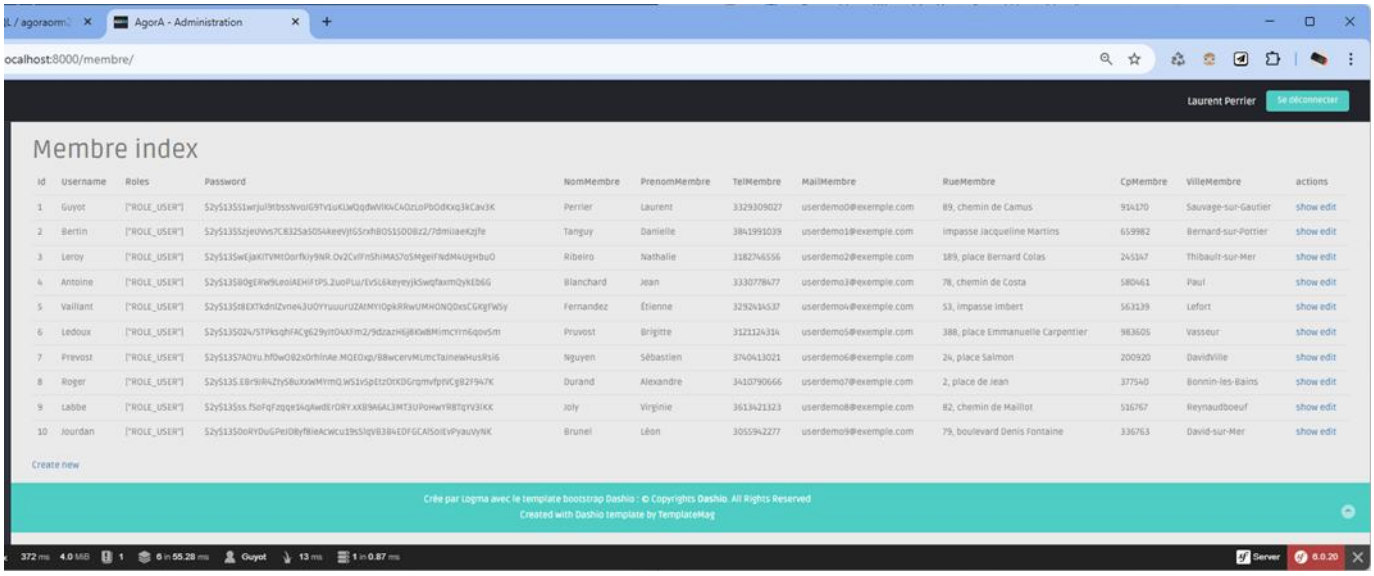
```
C:\Windows\System32\cmd.e

C:\wamp64\www\agoram61>php bin/console make:crud Membre

 Choose a name for your controller class (e.g. MembreController) [MembreController]:
 > Membre

 Do you want to generate tests for the controller?. [Experimental] (yes/no) [no]:
 >

 created: src/Controller/MembreController.php
 created: src/Form/MembreType.php
 created: templates/membre/_delete_form.html.twig
 created: templates/membre/_form.html.twig
 created: templates/membre/edit.html.twig
 created: templates/membre/index.html.twig
 created: templates/membre/new.html.twig
 created: templates/membre/show.html.twig

 Success!

 Next: Check your new CRUD by going to /membre/
```

2. Consultez les différents fichiers créés et notamment le contrôleur. Notez que le début de la route de chaque contrôleur est indiqué au niveau de la classe. A consulter pas à modifier
L'URL qui mène à la méthode index est : localhost:8000/membres

Fichier agoram61/src/Controller/MembresController.php

```php
. . .
#[Route('/membre')]
class MembreController extends AbstractController
{
    #[Route('/', name: 'app_membre_index', methods: ['GET'])]
    public function index(MembreRepository $membreRepository): Response
    {
        return $this->render('membre/index.html.twig', [
            'membre' => $membreRepository->findAll(),
        ]);
    }
. . .
```

3. Testez avec localhost:8000/membre.
Les membres sont affichés et la consultation détaillée d'un membre fonctionne (show) :



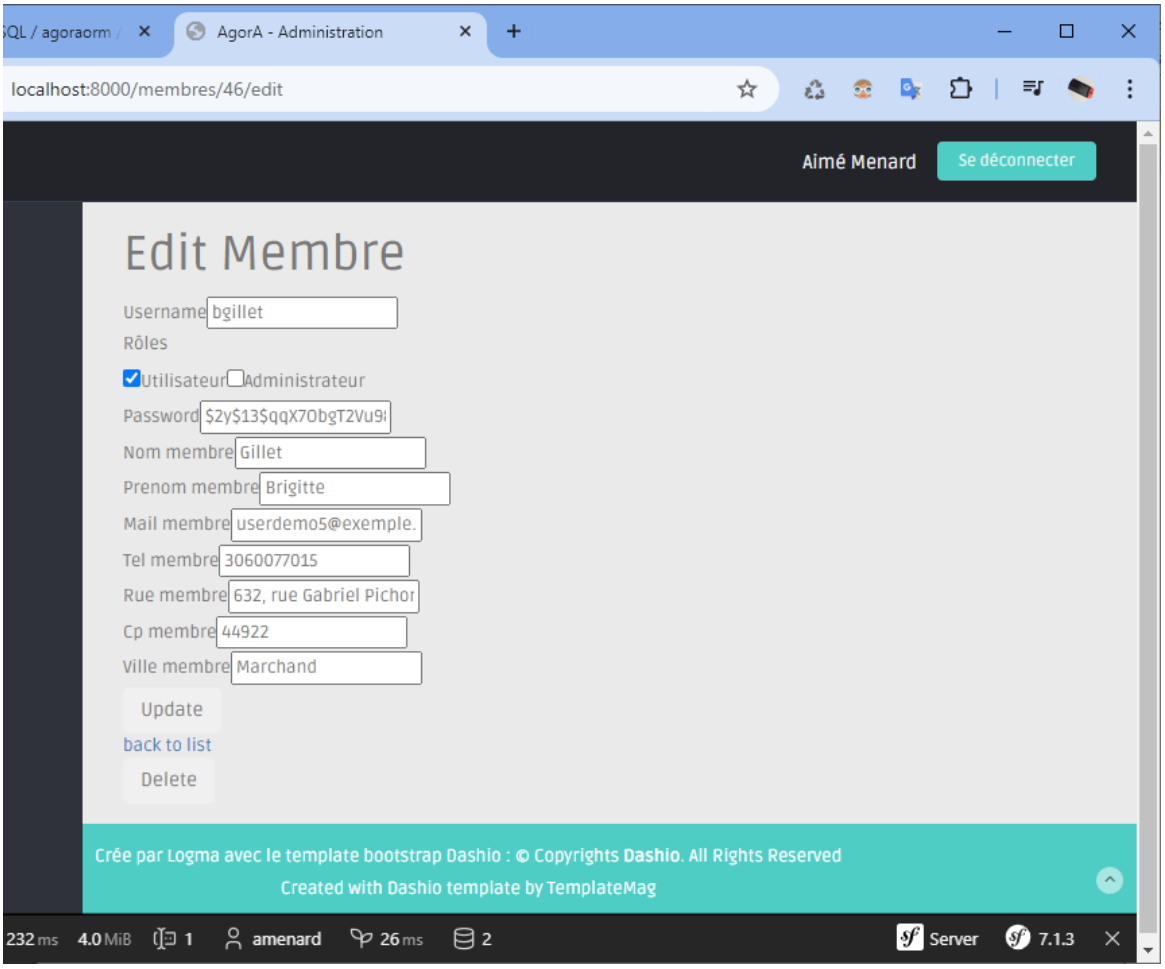
4. **Par contre, la modification et la création d'un membre ne fonctionnent pas.** C'est la propriété roles qui pose problème à cause de son type. Dans le formulaire type, il faut préciser comment afficher l'attribut roles. Comme c'est un tableau qui est rendu au formulaire, nous l'afficherons sous la forme de cases à cocher (une par rôle possible). Effectuez les modifications :

Fichier agoram61/src/Form/MembreType.php

```php
<?php

namespace App\Form;

use App\Entity\User;
use Symfony\Component\Form\AbstractType;
use Symfony\Component\Form\Extension\Core\Type\ChoiceType;
use Symfony\Component\Form\FormBuilderInterface;
use Symfony\Component\OptionsResolver\OptionsResolver;

class UserType extends AbstractType
{
    public function buildForm(FormBuilderInterface $builder, array $options): void
    {
        $builder
            ->add('username')
            ->add('roles', ChoiceType::class, [
                'choices' => [
                    'Utilisateur' => 'ROLE_USER',
                    'Administrateur' => 'ROLE_ADMIN'
                ],
                'expanded' => true,
                'multiple' => true,
                'label' => 'Rôles'
            ])
            ->add('password')
            ->add('nomMembre')
            ->add('prenomMembre')
            ->add('mailMembre')
            ->add('telMembre')
            ->add('rueMembre')
```

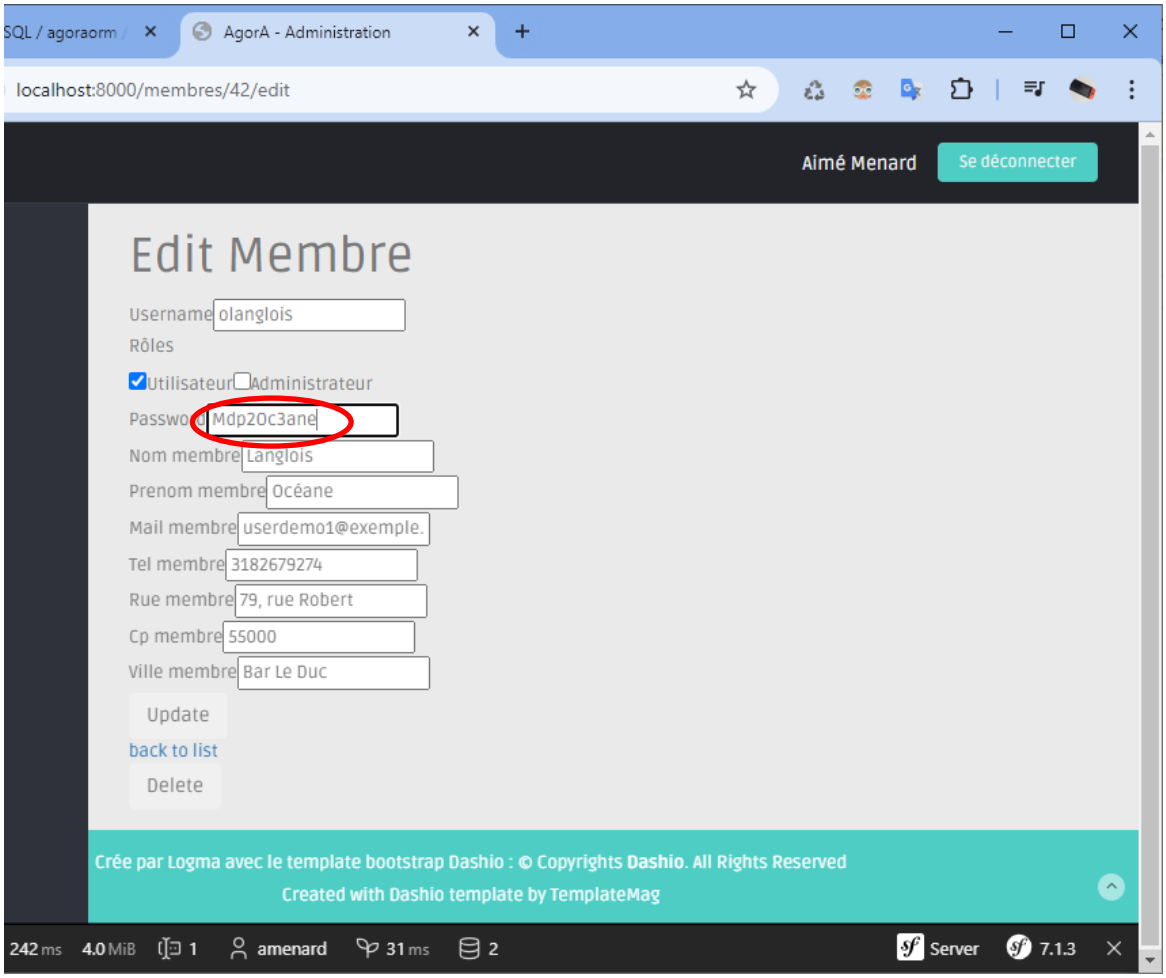
```
            ->add('cpMembre')
            ->add('villeMembre');        ;
    }

    public function configureOptions(OptionsResolver $resolver): void
    {
        $resolver->setDefaults([
            'data_class' => Membre::class,
        ]);
    }
}
```

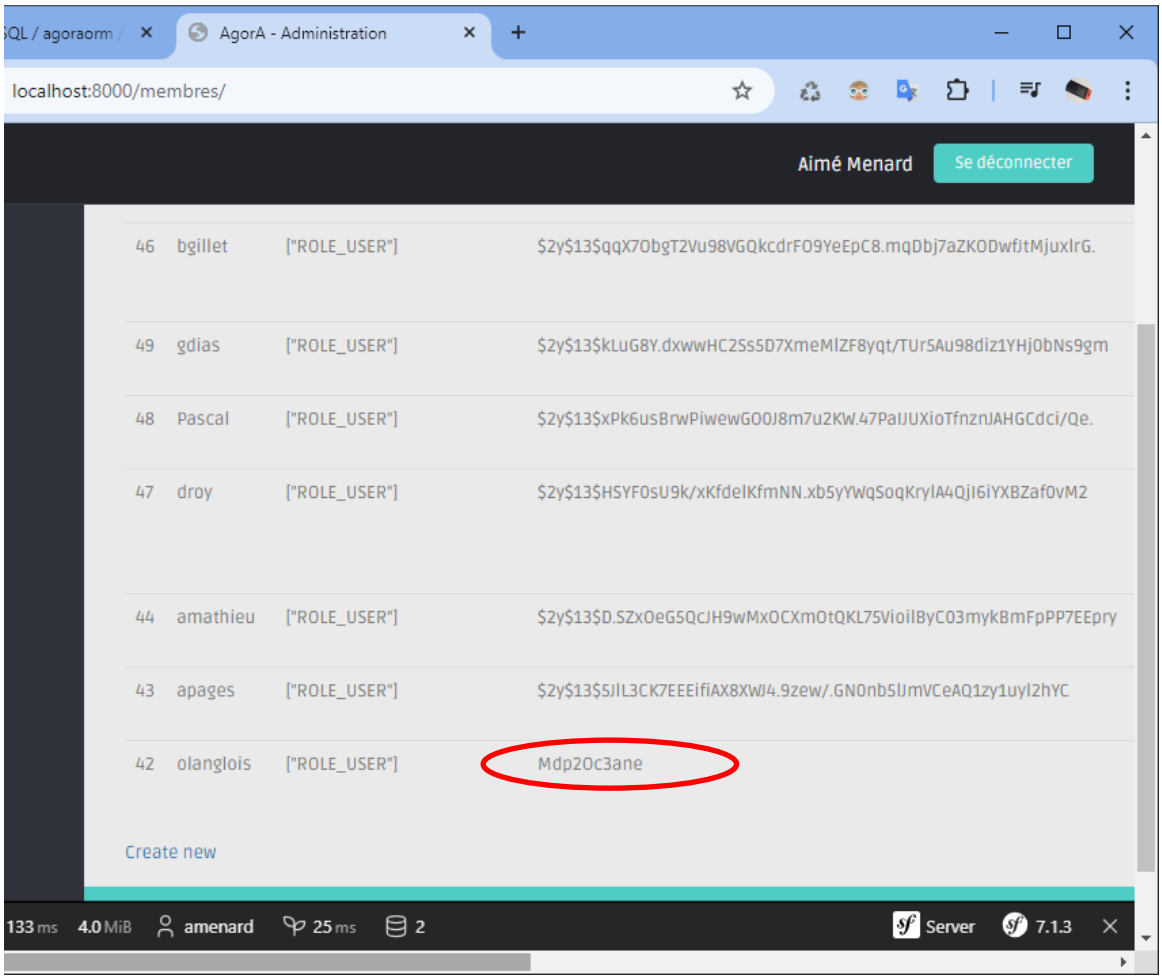
5. Testez avec localhost:8000/membre et demandez la modification d'un membre (edit)



6. Il faut aussi hacher le mot de passe

Si on ne modifie pas le code, le mot de passe ne sera pas haché, il sera enregistré tel que depuis les formulaires de création et de modification.
Pour le vérifier, créer un nouveau membre et constater que le mot de passe est sauvegardé en clair :


SQL / agoraorm
AgorA - Administration
localhost:8000/membres/42/edit
Aimé Menard
Se déconnecter
Edit Membre
Username olanglois
Rôles
Utilisateur Administrateur
Password Mdp2Oc3ane
Nom membre Langlois
Prenom membre Océane
Mail membre userdemo1@exemple.
Tel membre 3182679274
Rue membre 79, rue Robert
Cp membre 55000
Ville membre Bar Le Duc
Update
back to list
Delete
Crée par Logma avec le template bootstrap Dashio : © Copyrights Dashio. All Rights Reserved
Created with Dashio template by TemplateMag
242 ms 4.0 MiB 1 amenard 31 ms 2 Server 7.1.3


SQL / agoraorm
AgorA - Administration
localhost:8000/membres/
Aimé Menard
Se déconnecter
46 bgillet ["ROLE_USER"] $2y$13$qqX7ObgT2Vu98VGQkcdrFO9YeEpC8.mqDbj7aZKODwfJtMjuxlrG.
49 gdias ["ROLE_USER"] $2y$13$kLuG8Y.dxwwHC2Ss5D7XmeMlZF8yqt/TUr5Au98diz1YHj0bNs9gm
48 Pascal ["ROLE_USER"] $2y$13$xPk6usBrwPiwewGO0J8m7u2KW.47PaIJUXioTfnznJAHGCdci/Qe.
47 droy ["ROLE_USER"] $2y$13$HSYFOsU9k/xKfdeIKfmNN.xb5yYWqSoqKrylA4QjI6iYXBZaf0vM2
44 amathieu ["ROLE_USER"] $2y$13$D.SZxOeG5QCJH9wMxOCXmOtQKL75VioilByC03mykBmFpPP7EEpry
43 apages ["ROLE_USER"] $2y$13$5JIL3CK7EEEifiAX8XWJ4.9zew/.GNOnb5lJmVCeAQ1zy1uyl2hYC
42 olanglois ["ROLE_USER"] Mdp2Oc3ane
Create new
133 ms 4.0 MiB amenard 25 ms 2 Server 7.1.3

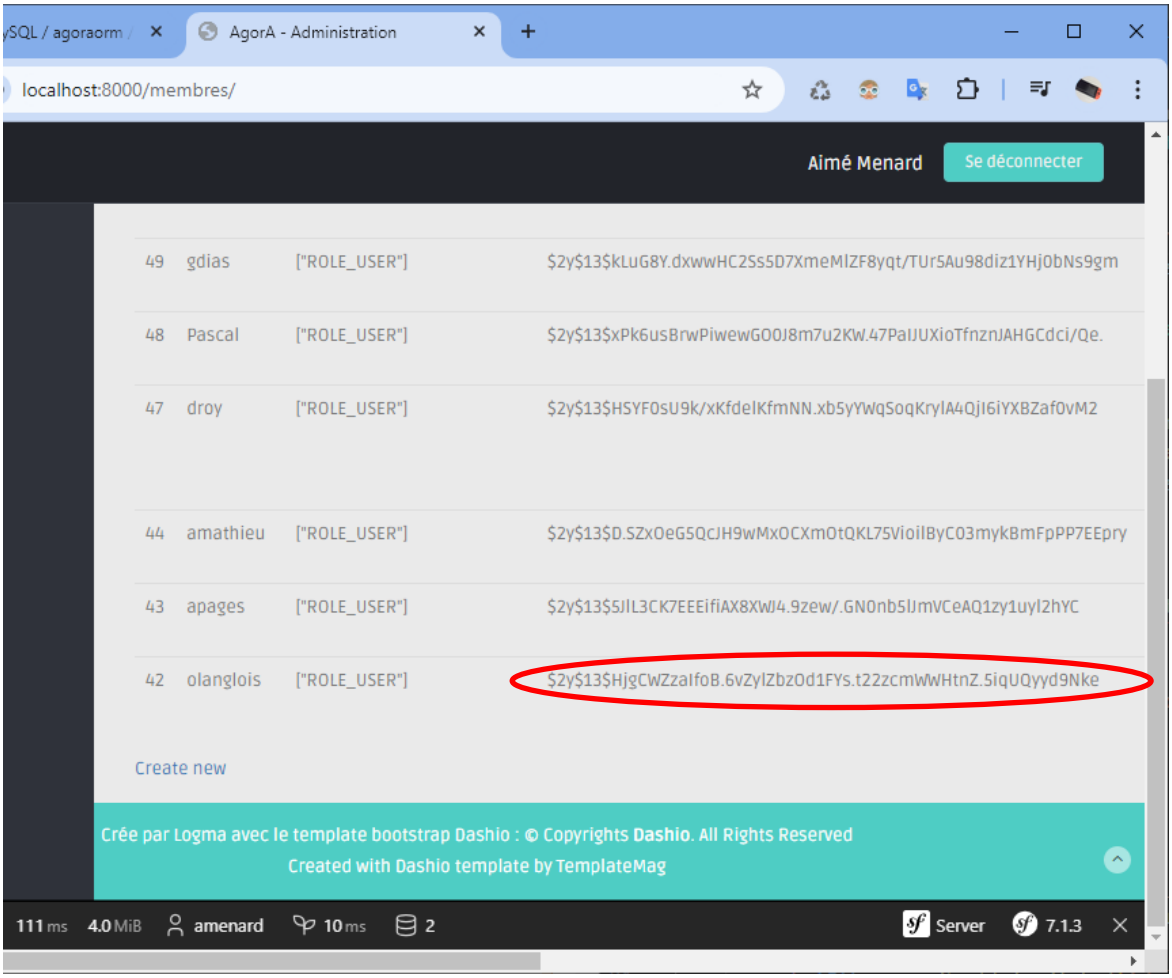
Pour faire évoluer cela, il faut modifier le fichier MembreController.php

Fichier agora5/src/Controller/MembreController.php

```php
<?php

namespace App\Controller;

use App\Entity\User;
use App\Form\UserType;
use App\Repository\UserRepository;
use Doctrine\ORM\EntityManagerInterface;
use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
use Symfony\Component\HttpFoundation\Request;
use Symfony\Component\HttpFoundation\Response;
use Symfony\Component\Routing\Annotation\Route;
use Symfony\Component\PasswordHasher\Hasher\UserPasswordHasherInterface;

#[Route('/membres')]
class MembresController extends AbstractController
{
    #[Route('/', name: 'app_membres_index', methods: ['GET'])]
    public function index(MembreRepository $membreRepository): Response
    {
        return $this->render('membre/index.html.twig', [
            'users' => $membreRepository->findAll(),
        ]);
    }

    #[Route('/new', name: 'app_membres_new', methods: ['GET', 'POST'])]
    public function new(Request $request, EntityManagerInterface $entityManager,
UserPasswordHasherInterface $unPasswordHasher): Response
    {
        $user = new Membre();
        $form = $this->createForm(MembreType::class, $membre);
        $form->handleRequest($request);

        if ($form->isSubmitted() && $form->isValid()) {
            $passwordSaisi = $membre->getPassword();
            $membre->setPassword($unPasswordHasher->hashPassword($membre, $passwordSaisi));

            $entityManager->persist($user);
            $entityManager->flush();

            return $this->redirectToRoute('app_membres_index', [],
Response::HTTP_SEE_OTHER);
        }

        return $this->render('membres/new.html.twig', [
            'membre' => $membre,
            'form' => $form,
        ]);
    }
```

```
    #[Route('/{id}', name: 'app_membres_show', methods: ['GET'])]
    public function show(User $membre): Response
    {
        return $this->render('membres/show.html.twig', [
            'membre' => $membre,
        ]);
    }

    #[Route('/{id}/edit', name: 'app_membres_edit', methods: ['GET', 'POST'])]
    public function edit(Request $request, Membre $membre, EntityManagerInterface
$entityManager, UserPasswordHasherInterface $unPasswordHasher): Response
    {
        $form = $this->createForm(UserType::class, $membre);
        $form->handleRequest($request);

        if ($form->isSubmitted() && $form->isValid()) {
            $passwordSaisi = $membre->getPassword();
            $membre->setPassword($unPasswordHasher->hashPassword($membre, $passwordSaisi));
            $entityManager->flush();

            return $this->redirectToRoute('app_membres_index', [],
Response::HTTP_SEE_OTHER);
        }

        return $this->render('membres/edit.html.twig', [
            'membre' => $membre,
            'form' => $form,
        ]);
    }

    #[Route('/{id}', name: 'app_membres_delete', methods: ['POST'])]
    public function delete(Request $request, Membre $membre,
EntityManagerInterface $entityManager): Response
    {
        if ($this->isCsrfTokenValid('delete'.$membre->getId(), $request->request-
>get('_token'))) {
            $entityManager->remove($membre);
            $entityManager->flush();
        }

        return $this->redirectToRoute('app_membres_index', [],
Response::HTTP_SEE_OTHER);
    }
}
```

Une fois ces modifications apportées, l'application va chiffrer le mot de passe avant de l'enregistrer, que ce soit depuis le formulaire de creation que depuis celui de modification.
Rééditer l'utilisateur précédemment créé et qui a un mot de passe en clair et appuyer sur "save". Constater que le mot de passe est alors haché :



Modifier le menu “Membres” pour appeler la route localhost:8000/membre :

```
C: > wamp64 > www > agoram61 > templates > menu.html.twig
8   <div id="sidebar" class="nav-collapse">
10      <ul class="sidebar-menu" id="nav-accordion">
14          <li class="sub-menu">
19              <ul class="sub">
24                  <li><a href="{{path('app_pegi_index')}}">Pegi</a></li>
25              </ul>
26          </li>
27          <li class="sub-menu">
28              <a{% if menuActif is defined and menuActif == 'Tournois' %} class="active" {% endif %} href="javascrip
29                  <i class="fa fa-group"></i>
30                  <span>Tournois</span>
31              </a>
32              <ul class="sub">
33                  <li><a href="{{path('app_tournoi_index')}}">Tournois</a></li>
34                  <li><a href="{{path('app_cat_tournois_index')}}">Catégorie de tournois</a></li>
35                  <li><a href="{{path('app_participant_index')}}">Participants</a></li>
36                  <li><a href="{{path('app_stat_sql_tournoi')}}">Dashboord tournois (SQL)</a></li>
37                  <li><a href="{{path('app_stat_dql_tournoi')}}">Dashboord tournois (DQL)</a></li>
38                  <li><a href="{{path('app_stat_qb_tournoi')}}">Dashboord tournois (QB)</a></li>
39                  <li><a href="{{path('app_stat_tournoi_index')}}">Dashboord tournois chart.js</a></li>
40              </ul>
41          </li>
42          <li class="sub-menu">
43              <a{% if menuActif is defined and menuActif == 'Membres' %} class="active" {% endif %} href="javascript
44                  <i class="fa fa-user"></i>
45                  <span>Membres</span>
46              </a>
47              <ul class="sub">
48                  <li><a href="{{path('app_membre_index')}}">Membres</a></li>
49              </ul>
50          </li>
51          {# {% endif %} #}
52      </ul>
53      <!-- sidebar menu end-->
54  </div>
55  </aside>
56  <!--sidebar end-->
57  {# {% endblock %}#}
58
```

Modifier l'appel de la vue membre/index.html/twig en veillant à passer le parameter menuActif à "membress" pour que le menu reste deplié :

```
C: > wamp64 > www > agoram61 > src > Controller > MembreController.php
6   use App\Form\MembreType;
7   use App\Repository\MembreRepository;
8   use Doctrine\ORM\EntityManagerInterface;
9   use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
10  use Symfony\Component\HttpFoundation\Request;
11  use Symfony\Component\HttpFoundation\Response;
12  use Symfony\Component\Routing\Annotation\Route;
13  use Symfony\Component\PasswordHasher\Hasher\UserPasswordHasherInterface;
14
15  #[Route('/membre')]
16  class MembreController extends AbstractController
17  {
18      #[Route('/', name: 'app_membre_index', methods: ['GET'])]
19      public function index(MembreRepository $membreRepository): Response
20      {
21          return $this->render('membre/index.html.twig', [
22              'membres' => $membreRepository->findAll(), 'menuActif' => 'Membres',
23          ]);
24      }
25
26      #[Route('/new', name: 'app_membre_new', methods: ['GET', 'POST'])]
```

2.6 Gérer les autorisations

Rappel : **S'authentifier** c'est confirmer son identité en fournissant par exemple un nom d'utilisateur ou un email et un mot de passe sur un système.

Les **autorisations** permettent d'interdire ou d'autoriser l'accès à certaines pages. En s'authentifiant on décline son identité et en fonction de cette identité on est autorisé ou non à accéder à certaines pages.

Dans la pratique, il y a différents profils d'utilisateurs (administrateur, utilisateur, …) avec des autorisations personnalisées. Un profil utilisateur correspond à un **rôle**; par exemple le rôle ROLE_ADMIN

Nous nous contenterons à ce stade, de 2 rôles :
- ROLE_ADMIN : administrateur , aura un accès total
- ROLE_USER : utilisateur , aura un accès à toutes les pages sauf celles de la gestion des catégories

1. Définir les autorisations dans security.yaml :

Fichier agoram61/config/packages/security.yaml

```yaml
. . .

# Easy way to control access for large sections of your site
# Note: Le premier access control qui correspond est appliqué
access_control:
    #  Un utilisateur anonyme a uniquement accès à la page de connexion (route /login)
    # correspond à /login/*
    - { path: '^/login', roles: PUBLIC_ACCESS }

    #  Un administrateur a accès à la gestion des catégories
    # correspond à /membres/* sauf si la règle précédente s'applique
    - { path: '^/membres', roles: ROLE_ADMIN }

    #  Un utilisateur a accès à tout sauf la gestion des catégories
    # correspond à /* sauf si la règle précédente s'applique
    - { path: '^/*', roles: [ROLE_ADMIN, ROLE_USER] }
```

2. Attribuez les rôles aux membres

Par défaut, chaque utilisateur existant a au moins le rôle ROLE_USER. C'est déjà fait dans la classe Membre.php

```php
/**
 * @see UserInterface
 */
public function getRoles(): array
{
    $roles = $this->roles;
    // guarantee every user at least has ROLE_USER
    $roles[] = 'ROLE_USER';

    return array_unique($roles);
}
```

Nous allons attribuer le rôle ROLE_ADMIN à un des membres. Par exemple, le premier qui a été créé par faker, via phpMyAdmin (ou MysqlWorkbench ou autre) modifiez l'attribut roles de l'utilisateur. On lui affecte les rôles **["ROLE_USER", "ROLE_ADMIN"]**

id	email	roles	password	nom	prenom	telephone
1	userdemo0@exemple.com	["ROLE_USER", "ROLE_ADMIN"]	$2y$13$9tN055JdwWSJ4KiGB8YmpeMKqW/VcWXaZO3vlosSu0u...	Dupont0	Jean0	0612345670
2	userdemo1@exemple.com	[]	$2y$13$l.JdZOedcR0prckcr9gbUuWm2o20bDNmmGXdsD0nw1....	Dupont1	Jean1	0612345671
3	userdemo2@exemple.com	[]	$2y$13$pvln2rgDz7lnrXN7gll7put9o9/ACyenWiUCj4ykJka...	Dupont2	Jean2	0612345672
4	userdemo3@exemple.com	[]	$2y$13$yzKvkdoslVw.24Jz.7DT0.U1lQauudlAGI8DwDQzkfz...	Dupont3	Jean3	0612345673

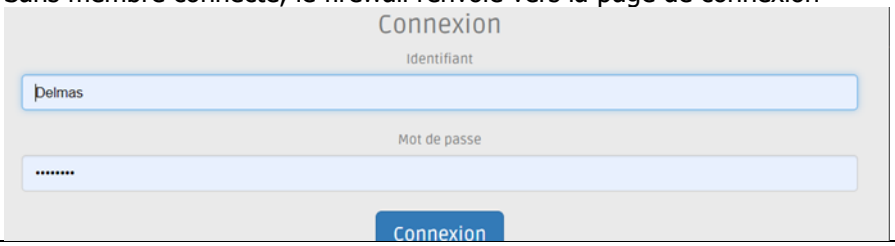
3. Réserver l'accès au menu Membres pour le rôle administrateur

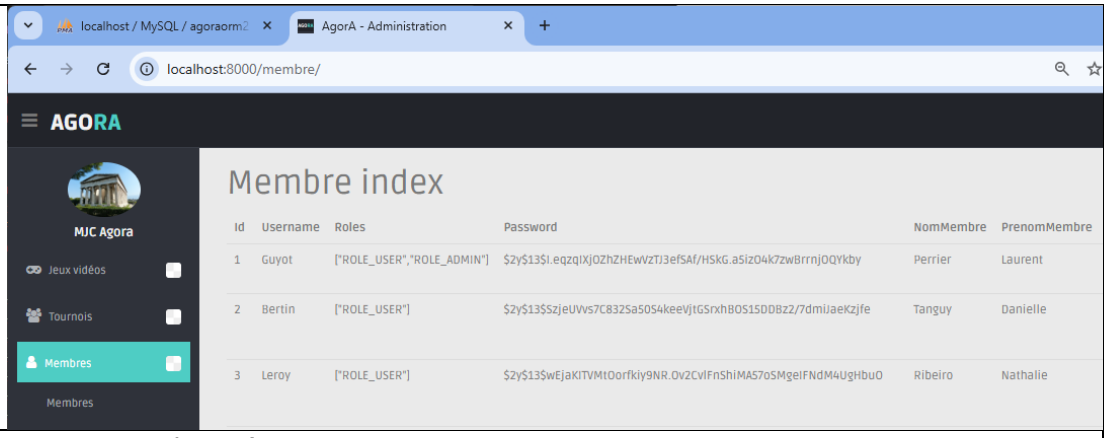
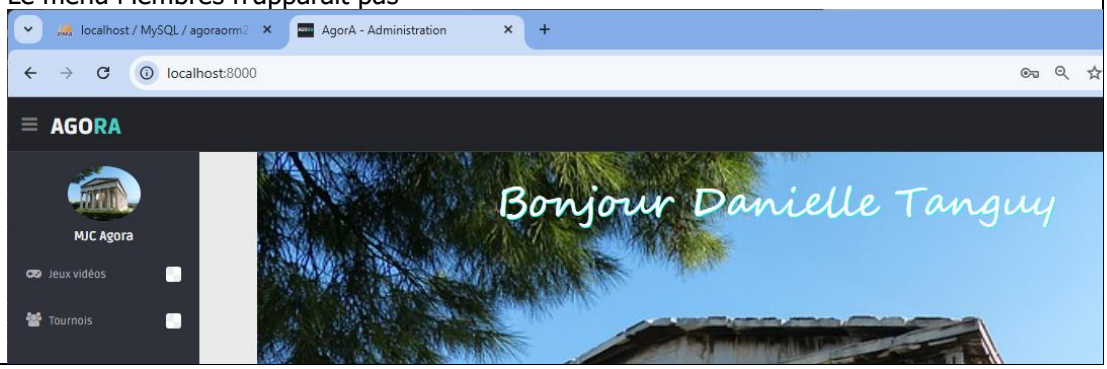
Modifier le menu pour accéder au formulaire de gestion des membres. Conditionner son affichage au fait que l'utilisateur connecté à le rôle ADMIN.

Fichier agora5/templates/menu.html.twig

```
...
            </ul>
        </li>
        {% if is_granted('ROLE_ADMIN') %}
        <li class="sub-menu">
            <a{% if menuActif is defined and menuActif == 'Membres' %} class="active" {% endif %} ... >
                <i class="fa fa-user"></i>
                <span>Membres</span>
            </a>
            <ul class="sub">
                <li><a href="{{path('app_membre_index')}}">Membres</a></li>
            </ul>
        </li>
        {% endif %}
    </ul>
    <!-- sidebar menu end-->
</div>
</aside>
<!--sidebar end-->
{# {% endblock %}#}
```

4. Tester

aucun membre connecté localhost:8000/membre	Sans membre connecté, le firewall renvoie vers la page de connexion 
membre connecté avec rôle ROLE_ADMIN localhost:8000/membre	Affichage du formulaire de gestion des membres et du menu complet.

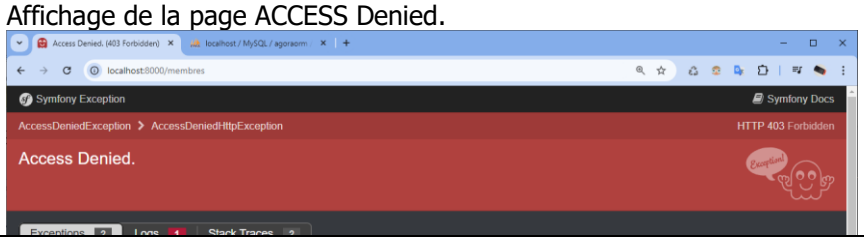
	
Membre connecté à l'application sans rôle ROLE_ADMIN	Le menu Membres n'apparait pas 

5. Gestion des accès au niveau des contrôleurs

Il est également possible de gérer les autorisations au niveau d'un contrôleur.

Pour autoriser à un utilisateur authentifié toutes les méthodes (contrôleurs) d'une classe ou une méthode en particulier, en ajoute au-dessus de la classe ou de la méthode :
#[IsGranted('IS_AUTHENTICATED_FULLY')]

Tester :

membre connecté sans rôle ROLE_ADMIN Comme cet utilisateur n'a pas le menu Membres, saisir directement la route localhost:8000/membre	Affichage de la page ACCESS Denied. 

3. Conclusion

Vous savez désormais :

- comment créer une entité spécifique destinée à être utilisée pour la connexion ,
- installer le composant Security et paramétrer le firewall pour qu'il prenne en compte cette entité spécifique,
- créer des fixtures pour remplir artificiellement une table de données fictive,
- modifier les formulaires pour gérer la phase de connexion, et de déconnection,
- gérer les membres et leurs rôles,
- gérer les droits d'accès tant au niveau du menu qu'au niveau des routes dans les contrôleurs.